SAAS Churn Analysis [Data Analytics Project]

Problem Stattement

Customer churn is one of the biggest challenges for SaaS businesses, as losing existing customers directly impacts recurring revenue and long-term growth. Companies often struggle to identify the key reasons why customers discontinue subscriptions, such as low product usage, poor customer satisfaction, billing patterns, service issues, or plan downgrades.

This project aims to analyze customer subscription and behavioral data to identify the major factors contributing to customer churn in a SaaS business. By examining variables such as subscription plans, product usage, customer support interactions, billing frequency, satisfaction scores, renewal behavior, and account activity, the project seeks to uncover churn patterns and provide actionable insights to improve customer retention, reduce revenue loss, and enhance customer experience.

Objective of the Project:

- Analyze customer churn trends and patterns in a SaaS environment.
- Identify key factors influencing churn behavior.
- Study the impact of customer engagement, subscription plans, and billing behavior on retention.
- Examine relationships between support tickets, satisfaction scores, and churn likelihood.
- Provide business recommendations to improve retention and reduce churn rate.

I. Loading Modules

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

II. Loading the CSV File.

```
df= pd.read_csv('/content/Churn Analysis Final.csv')
df
```

	Account_id	Subscription_id	Account_name	Industry	Country	Signup_date	Start_date	End_date	Refe
0	A-779e4e	S-92f228	Company_193	HealthTech	US	1/2/2023	1/9/2023	12/31/2024	
1	A-af0cd2	S-50aaaa	Company_204	Cybersecurity	IN	1/3/2023	1/12/2023	12/31/2024	
2	A-779e4e	S-18356d	Company_193	HealthTech	US	1/2/2023	1/28/2023	12/31/2024	
3	A-7df7a7	S-6676fa	Company_125	DevTools	US	1/28/2023	2/5/2023	12/31/2024	
4	A-d792a6	S-47ce2d	Company_486	DevTools	US	1/23/2023	2/6/2023	12/31/2024	
...	...	...	...	...	...	...	...	...	
4217	A-524364	S-a33656	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024	
4218	A-524364	S-dee62f	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024	
4219	A-0f7d77	S-47048b	Company_457	FinTech	IN	10/28/2024	12/31/2024	12/31/2024	
4220	A-08e34e	S-6b534f	Company_32	EdTech	US	6/27/2024	12/31/2024	12/31/2024	
4221	A-0f6450	S-9c9700	Company_400	FinTech	US	12/27/2024	12/31/2024	12/31/2024	

4222 rows × 25 columns

III. Discovering Data.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4222 entries, 0 to 4221
Data columns (total 25 columns):
 #   Column               Non-Null Count  Dtype
```

```
---  ------                --------------  -----
 0   Account_id            4222 non-null   object
 1   Subscription_id       4222 non-null   object
 2   Account_name          4222 non-null   object
 3   Industry              4222 non-null   object
 4   Country               4222 non-null   object
 5   Signup_date           4222 non-null   object
 6   Start_date            4222 non-null   object
 7   End_date              4222 non-null   object
 8   Referral_source       4222 non-null   object
 9   Plan_tier_before      4222 non-null   object
 10  Upgrade_flag          4222 non-null   bool
 11  Downgrade_flag        4222 non-null   bool
 12  Plan_tier_after       4222 non-null   object
 13  Churn_flag            4222 non-null   bool
 14  Seats                 4222 non-null   int64
 15  Is_trial              4222 non-null   bool
 16  Auto_renewal          4222 non-null   bool
 17  Mrr_amount            4222 non-null   int64
 18  Arr_amount            4222 non-null   int64
 19  Billing_frequency     4222 non-null   object
 20  Usage_count           4222 non-null   int64
 21  Usage_duration_secs   4222 non-null   int64
 22  Error_count           4222 non-null   int64
 23  Total_tickets         4222 non-null   int64
 24  Avg_satisfaction      4222 non-null   float64
dtypes: bool(5), float64(1), int64(7), object(12)
memory usage: 680.4+ KB
```

```
df.shape
```

```
(4222, 25)
```

```
# Numerical descriptive statistics
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Seats	4222.0	25.036002	20.032505	1.0	12.00	20.0	32.0	163.0
Mrr_amount	4222.0	2685.634060	3569.430215	19.0	539.00	1244.5	3381.0	33830.0
Arr_amount	4222.0	32227.608716	42833.162580	228.0	6468.00	14934.0	40572.0	405960.0
Usage_count	4222.0	50.207958	23.484773	0.0	33.00	48.0	66.0	148.0
Usage_duration_secs	4222.0	15224.216485	8140.197656	0.0	9213.75	14357.0	20255.5	53465.0
Error_count	4222.0	2.817385	2.574371	0.0	1.00	2.0	4.0	16.0
Total_tickets	4222.0	3.976551	1.842191	0.0	3.00	4.0	5.0	11.0
Avg_satisfaction	4222.0	3.924420	0.580198	0.0	3.80	4.0	4.2	5.0

IV. Statstical Analysis

```
# Mean, Median, Mode, Std, Variance of Numerical columns.
for col in df.select_dtypes(include=np.number).columns:
    print(f"\n{col}:")
    print("Mean:", df[col].mean())
    print("Median:", df[col].median())
    print("Mode:", df[col].mode()[0])
    print("Min:", df[col].min())
    print("Max:", df[col].max())
    print("Standard Deviation:", df[col].std())
    print("Variance:", df[col].var())
```

```
Seats:
Mean: 25.03600189483657
Median: 20.0
Mode: 9
Min: 1
```

```
Max: 163
Standard Deviation: 20.03250454887826
Variance: 401.3012385008282

Mrr_amount:
Mean: 2685.634059687352
Median: 1244.5
Mode: 3184
Min: 19
Max: 33830
Standard Deviation: 3569.4302149799755
Variance: 12740832.059611995

Arr_amount:
Mean: 32227.608716248225
Median: 14934.0
Mode: 38208
Min: 228
Max: 405960
Standard Deviation: 42833.16257975962
Variance: 1834679816.5841196

Usage_count:
Mean: 50.207958313595455
Median: 48.0
Mode: 45
Min: 0
Max: 148
Standard Deviation: 23.484773114810437
Variance: 551.5345682541232

Usage_duration_secs:
Mean: 15224.216485078163
Median: 14357.0
Mode: 0
Min: 0
Max: 53465
Standard Deviation: 8140.197655999097
Variance: 66262817.878733195

Error_count:
Mean: 2.817385125532923
```

```
Median: 2.0
Mode: 0
Min: 0
Max: 16
Standard Deviation: 2.5743713420231615
Variance: 6.627387806630135

Total_tickets:
Mean: 3.9765513974419706
```

```
# Categorical columns
cat_cols = df.select_dtypes(include='object').columns
print("Categorical Columns:")
print(cat_cols)
```

```
Categorical Columns:
Index(['Account_id', 'Subscription_id', 'Account_name', 'Industry', 'Country',
       'Signup_date', 'Start_date', 'End_date', 'Referral_source',
       'Plan_tier_before', 'Plan_tier_after', 'Billing_frequency'],
      dtype='object')
```

```
# Summary Report of the Dataset.
print("Dataset Shape:", df.shape)
print("\nNumerical Columns:")
print(df.select_dtypes(include=np.number).columns.tolist())

print("\nCategorical Columns:")
print(df.select_dtypes(include='object').columns.tolist())

print("\nStatistical Summary:")
display(df.describe().T)

print("\nCategorical Summary:")
display(df.describe(include='object').T)
```

```
Dataset Shape: (4222, 25)

Numerical Columns:
['Seats', 'Mrr_amount', 'Arr_amount', 'Usage_count', 'Usage_duration_secs', 'Error_count', 'Total_tickets', 'Av

Categorical Columns:
['Account_id', 'Subscription_id', 'Account_name', 'Industry', 'Country', 'Signup_date', 'Start_date', 'End_date

Statistical Summary:
```

	count	mean	std	min	25%	50%	75%	max
Total_tickets	4222.0	3.[illegible]551	1.842191	0.0	3.00	4.0	5.0	11.0
Avg_satisfaction	4222.0	3.9[illegible]420	0.580198	0.0	3.80	4.0	4.2	5.0

```
#Frequency Counts & Percentages for Important Categories.
```

```
industry_counts = df['Industry'].value_counts()
industry_percentages = (df['Industry'].value_counts(normalize=True) * 100).round(2)

# Combine into a DataFrame for better display
industry_summary = pd.DataFrame({'Count': industry_counts,
                                 'Percentage': industry_percentages})
industry_summary
```

Industry	Count	Percentage
DevTools	984	23.31
FinTech	930	22.03
Cybersecurity	866	20.51
HealthTech	786	18.62
EdTech	656	15.54

Next steps: Generate code with `industry_summary` New interactive sheet

```
RS_counts = df['Referral_source'].value_counts()
RS_percentages = (df['Referral_source'].value_counts(normalize=True) * 100).round(2)
```

	count	unique	top	freq
Subscription_id	4222	4222	S-9c9700	1
Industry	4222	5	DevTools	984
Country	4222	7	US	2513
Account_name	4222	4222	Company_287	17
Account_id	4222	4222	A-8bde0c	17

```
# Combine into a DataFrame for better display
RS_summary = pd.DataFrame({'Count': RS_counts,
                           'Percentage': RS_percentages})
RS_summary
```

Billing_frequency	Count	Percentage
Referral_source		
Organic	1001	23.71
Other	858	20.32
Ads	838	19.85
Event	777	18.40
Partner	748	17.72

Next steps: Generate code with RS_summary | New interactive sheet

```
Country_counts = df['Country'].value_counts()
Country_percentages = (df['Country'].value_counts(normalize=True) * 100).round(2)

# Combine into a DataFrame for better display
Country_summary = pd.DataFrame({'Count': Country_counts,
                           'Percentage': Country_percentages})
Country_summary
```

Country	Count	Percentage
US	2513	59.52
UK	497	11.77
IN	424	10.04
AU	261	6.18
CA	178	4.22
DE	176	4.17
FR	173	4.10

Next steps: Generate code with `Country_summary` New interactive sheet

```
df['Plan_tier_before'].value_counts()
PTB_counts = df['Plan_tier_before'].value_counts()
PTB_percentages = (df['Plan_tier_before'].value_counts(normalize=True) * 100).round(2)

# Combine into a DataFrame for better display
PTB_summary = pd.DataFrame({'Count': PTB_counts,
                            'Percentage': PTB_percentages})
PTB_summary
```

Plan_tier_before	Count	Percentage
Pro	1486	35.2
Basic	1465	34.7
Enterprise	1271	30.1

Next steps: Generate code with PTB_summary New interactive sheet

```
df['Plan_tier_after'].value_counts()
PTA_counts = df['Plan_tier_after'].value_counts()
PTA_percentages = (df['Plan_tier_after'].value_counts(normalize=True) * 100).round(2)

# Combine into a DataFrame for better display
PTA_summary = pd.DataFrame({'Count': PTA_counts,
                            'Percentage': PTA_percentages})
PTA_summary
```

Plan_tier_after	Count	Percentage
Enterprise	1450	34.34
Pro	1416	33.54
Basic	1356	32.12

Next steps: Generate code with PTA_summary New interactive sheet

```
BF_counts = df['Billing_frequency'].value_counts()
BF_percentages = (df['Billing_frequency'].value_counts(normalize=True) * 100).round(2)

# Combine into a DataFrame for better display
BF_summary = pd.DataFrame({'Count': BF_counts,
                           'Percentage': BF_percentages})
BF_summary
```

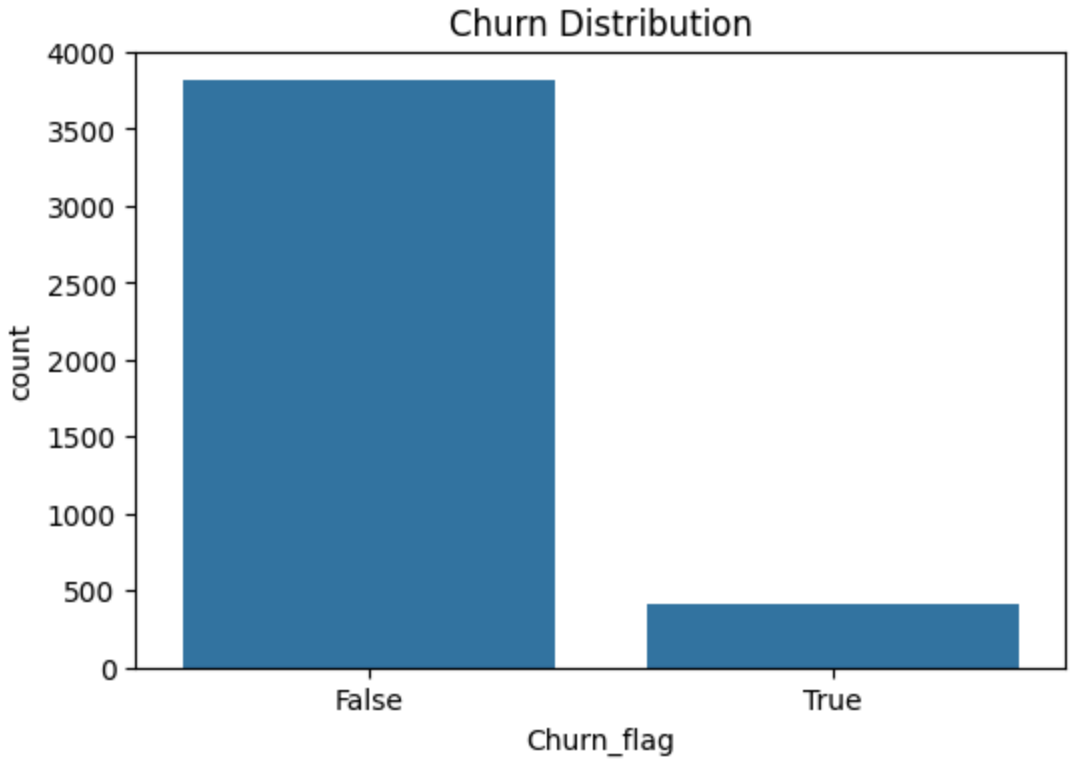
	Count	Percentage
Billing_frequency		
Monthly	2135	50.57
Annual	2087	49.43

Next steps: Generate code with BF_summary | New interactive sheet

```
# Churn distribution.
print(df['Churn_flag'].value_counts())

# Churn percentage.
churn_rate = (df['Churn_flag'].value_counts(normalize=True) * 100)

print("\nChurn Percentage:")
print(churn_rate)
```

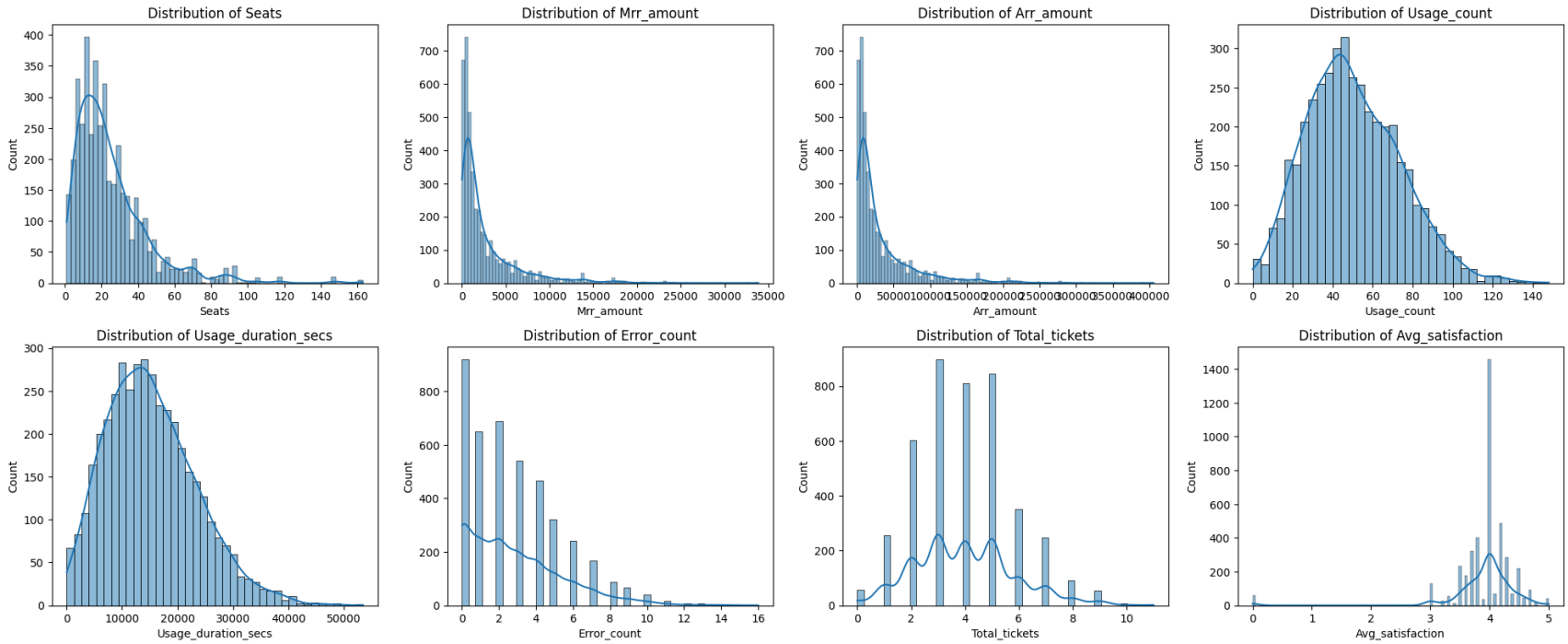
```
Churn_flag
False    3814
True      408
Name: count, dtype: int64

Churn Percentage:
Churn_flag
False    90.336333
True      9.663667
Name: proportion, dtype: float64
```

```
#Churn Visualization

plt.figure(figsize=(6,4))
sns.countplot(x='Churn_flag', data=df)
plt.title("Churn Distribution")
plt.show()
```



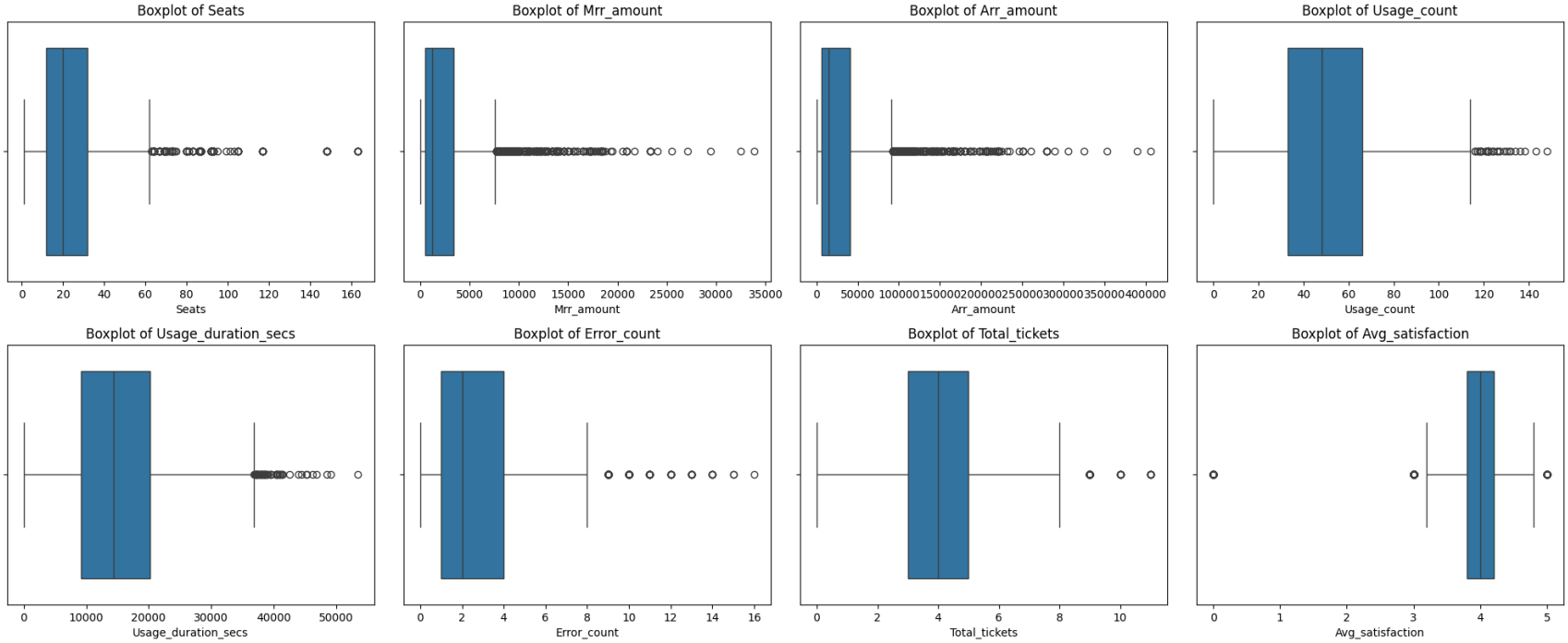

```
#  Distribution of Numerical Features.
num_cols = df.select_dtypes(include=np.number).columns
fig, axes = plt.subplots(4, 4, figsize=(20, 16))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    sns.histplot(df[col], kde=True, ax=axes[i])
    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')

# Remove empty subplots
for j in range(len(num_cols), len(axes)):
    fig.delaxes(axes[j])
```

```
plt.tight_layout()
plt.show()
```

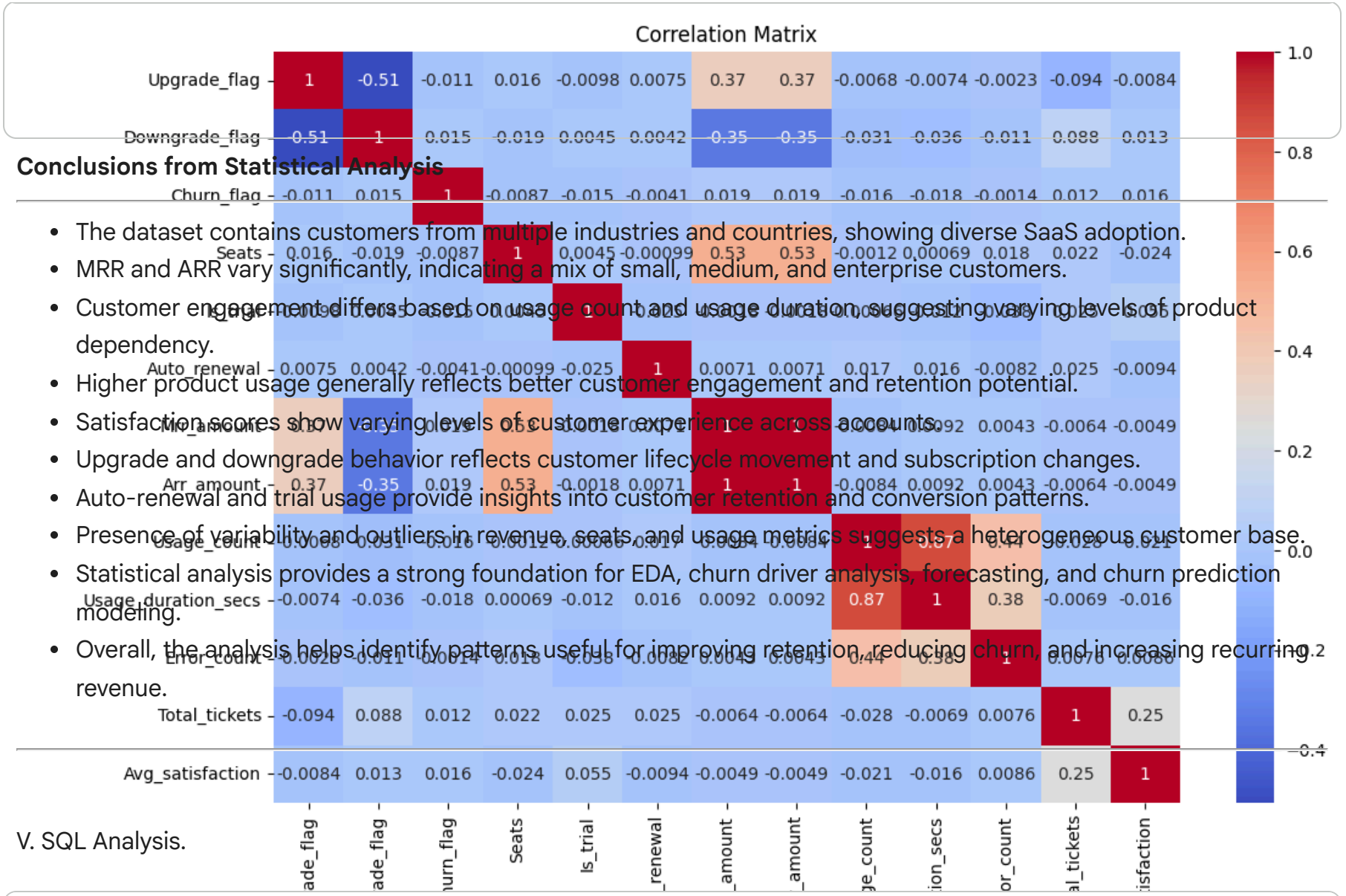



```
#Boxplot for Outlier Detection.
num_cols = df.select_dtypes(include=np.number).columns
fig, axes = plt.subplots(4, 4, figsize=(20, 16))
axes = axes.flatten()
```

```
for i, col in enumerate(num_cols):
    sns.boxplot(x=df[col],ax=axes[i])
    axes[i].set_title(f'Boxplot of {col}')
    axes[i].set_xlabel(col)

# Remove extra empty plots
for j in range(len(num_cols), len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()
```




```
#Correlation Matrix.
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



Conclusions from Statistical Analysis

- The dataset contains customers from multiple industries and countries, showing diverse SaaS adoption.
- MRR and ARR vary significantly, indicating a mix of small, medium, and enterprise customers.
- Customer engagement differs based on usage count and usage duration, suggesting varying levels of product dependency.
- Higher product usage generally reflects better customer engagement and retention potential.
- Satisfaction scores show varying levels of customer experience across accounts.
- Upgrade and downgrade behavior reflects customer lifecycle movement and subscription changes.
- Auto-renewal and trial usage provide insights into customer retention and conversion patterns.
- Presence of variability and outliers in revenue, seats, and usage metrics suggests a heterogeneous customer base.
- Statistical analysis provides a strong foundation for EDA, churn driver analysis, forecasting, and churn prediction modeling.
- Overall, the analysis helps identify patterns useful for improving retention, reducing churn, and increasing recurring revenue.

V. SQL Analysis.

```
#Importing Duckdb Module.

import duckdb
import pandas as pd
def result():
```

```
result= duckdb.sql(query)
print(result)
```

```
#1. View Dataset

query= 'SELECT * FROM df'
result()
```

Account_id varchar	Subscription_id varchar	Account_name varchar	Industry varchar	Country varchar	Signup_date varchar	Start_date varchar	End_date varchar
A-779e4e	S-92f228	Company_193	HealthTech	US	1/2/2023	1/9/2023	12/31/2024
A-af0cd2	S-50aaaa	Company_204	Cybersecurity	IN	1/3/2023	1/12/2023	12/31/2024
A-779e4e	S-18356d	Company_193	HealthTech	US	1/2/2023	1/28/2023	12/31/2024
A-7df7a7	S-6676fa	Company_125	DevTools	US	1/28/2023	2/5/2023	12/31/2024
A-d792a6	S-47ce2d	Company_486	DevTools	US	1/23/2023	2/6/2023	12/31/2024
A-779e4e	S-11066a	Company_193	HealthTech	US	1/2/2023	2/12/2023	11/24/2023
A-f17767	S-7de656	Company_123	DevTools	US	1/30/2023	2/15/2023	8/25/2023
A-b11cd0	S-7a36ee	Company_439	DevTools	US	1/20/2023	2/16/2023	6/22/2024
A-d792a6	S-78c894	Company_486	DevTools	US	1/23/2023	2/17/2023	12/31/2024
A-977ca0	S-ea5984	Company_232	HealthTech	IN	2/20/2023	2/20/2023	12/31/2024
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
A-463db0	S-3eb56d	Company_16	HealthTech	US	12/5/2024	12/31/2024	12/31/2024
A-94f8c9	S-1eb467	Company_51	HealthTech	US	3/29/2024	12/31/2024	12/31/2024
A-0f6450	S-e11cde	Company_400	FinTech	US	12/27/2024	12/31/2024	12/31/2024
A-524364	S-47b86b	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024
A-524364	S-7480e6	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024
A-524364	S-a33656	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024
A-524364	S-dee62f	Company_388	DevTools	FR	12/31/2024	12/31/2024	12/31/2024
A-0f7d77	S-47048b	Company_457	FinTech	IN	10/28/2024	12/31/2024	12/31/2024
A-08e34e	S-6b534f	Company_32	EdTech	US	6/27/2024	12/31/2024	12/31/2024
A-0f6450	S-9c9700	Company_400	FinTech	US	12/27/2024	12/31/2024	12/31/2024

4222 rows (20 shown)

```
#2. Total Number of Customers

query= 'SELECT COUNT(*) AS total_customers FROM df'
result()
```

```
┌─────────────────┐
│ total_customers │
│      int64      │
├─────────────────┤
│            4222 │
└─────────────────┘
```

```
#3. Distinct Subscription Plans Before.

query= '''SELECT Plan_tier_before, COUNT(*) AS customer_count FROM df
          GROUP BY Plan_tier_before ORDER BY customer_count DESC'''
result()
```

```
┌──────────────────┬──────────────────┐
│ Plan_tier_before │ customer_count   │
│     varchar      │      int64       │
├──────────────────┼──────────────────┤
│ Pro              │             1486 │
│ Basic            │             1465 │
│ Enterprise       │             1271 │
└──────────────────┴──────────────────┘
```

```
#4. Distinct Subscription Plans After.

query= '''SELECT Plan_tier_after, COUNT(*) AS customer_count FROM df
          GROUP BY Plan_tier_after ORDER BY customer_count DESC'''
result()
```

```
┌─────────────────┬──────────────────┐
│ Plan_tier_after │ customer_count   │
│     varchar     │      int64       │
├─────────────────┼──────────────────┤
```

Enterprise	1450
Pro	1416
Basic	1356

#5. Calculate Churn Rate.

```
query= '''SELECT Churn_flag, COUNT(*) AS total_customers,
          (COUNT(*) * 100.0 / (SELECT COUNT(*) FROM df)) AS churn_rate
          FROM df GROUP BY Churn_flag'''
result()
```

Churn_flag boolean	total_customers int64	churn_rate double
false	3814	90.33633349123637
true	408	9.663666508763619

#6. Average Monthly Revenue (MRR).

```
query= '''SELECT ROUND(AVG(Mrr_amount),2) AS Avg_MRR FROM df'''
result()
```

Avg_MRR double
2685.63

```
#7. Average Annual Revenue (ARR).

query= '''SELECT ROUND(AVG(Arr_amount),2) AS Avg_ARR FROM df'''
result()
```

Avg_ARR
double
32227.61

```
#8. Top 10 Highest Monthly Revenue Customers.

query= '''SELECT Account_id, Subscription_id, Industry, Mrr_amount FROM df
          ORDER BY Mrr_amount DESC LIMIT 10'''
result()
```

Account_id	Subscription_id	Industry	Mrr_amount
varchar	varchar	varchar	int64
A-c70870	S-78f738	Cybersecurity	33830
A-56962b	S-e98121	EdTech	32437
A-18793f	S-155b3e	EdTech	29452
A-eb7fac	S-4c100f	DevTools	27064
A-118f1c	S-9f8b23	Cybersecurity	25472
A-68f37c	S-0f3df6	HealthTech	24079
A-d4e0d4	S-f11653	FinTech	23283
A-d4e0d4	S-7904d3	FinTech	23283
A-d4e0d4	S-fe0869	FinTech	23283
A-d4e0d4	S-61cc6f	FinTech	23283
10 rows			4 columns

```
#9. Top 10 Highest Annual Revenue Customers.
```

```
query= '''SELECT Account_id, Subscription_id, Industry, Arr_amount FROM df
          ORDER BY Arr_amount DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Industry varchar	Arr_amount int64
A-c70870	S-78f738	Cybersecurity	405960
A-56962b	S-e98121	EdTech	389244
A-18793f	S-155b3e	EdTech	353424
A-eb7fac	S-4c100f	DevTools	324768
A-118f1c	S-9f8b23	Cybersecurity	305664
A-68f37c	S-0f3df6	HealthTech	288948
A-d4e0d4	S-f11653	FinTech	279396
A-d4e0d4	S-7904d3	FinTech	279396
A-d4e0d4	S-fe0869	FinTech	279396
A-d4e0d4	S-61cc6f	FinTech	279396
10 rows			4 columns

#10. Industry-wise Customer Distribution.

```
query= '''SELECT Industry, count(*) as Total_customers FROM df GROUP BY Industry
          ORDER BY Total_customers DESC'''
result()
```

Industry varchar	Total_customers int64
DevTools	984
FinTech	930
Cybersecurity	866
HealthTech	786
EdTech	656

```
#11. Average Satisfaction Score by Plan.

query= '''SELECT Plan_tier_after, round(avg(Avg_satisfaction),3) as Avg_satisfaction
          FROM df GROUP BY Plan_tier_after ORDER BY Avg_satisfaction DESC'''
result()
```

Plan_tier_after varchar	Avg_satisfaction double
Enterprise	3.936
Basic	3.933
Pro	3.905

```
#12. Customers with High Support Tickets.

query= '''SELECT Account_id, Subscription_id, Total_tickets FROM df
          order by Total_tickets desc limit 10'''
result()
```

Account_id varchar	Subscription_id varchar	Total_tickets int64
A-bb3bd4	S-342037	11
A-bb3bd4	S-550202	11
A-bb3bd4	S-97aa94	11
A-bb3bd4	S-7d66fb	11
A-bb3bd4	S-9eb2c5	11
A-bb3bd4	S-a1676a	11
A-3ce5b8	S-dc7143	10
A-3ce5b8	S-e32bf8	10
A-3ce5b8	S-e11518	10
A-3ce5b8	S-c48d0d	10
10 rows		3 columns

```
#13. Revenue by Billing Frequency.

query= '''SELECT Billing_frequency, SUM(Arr_amount) AS Total_revenue FROM df
          GROUP BY Billing_frequency'''
result()
```

Billing_frequency varchar	Total_revenue int128
Annual	67168776
Monthly	68896188

```
#14. Country-wise Revenue.

query= '''SELECT country, ROUND(SUM(Arr_amount),3) AS Total_Arr FROM df
          GROUP BY country ORDER BY Total_Arr DESC'''
result()
```

Country varchar	Total_Arr int128
US	79615140
UK	14645652
IN	14331900
AU	10079760
FR	7077552
CA	5730744
DE	4584216

```
#15. Average Product Usage by Churn Status.

query= '''SELECT Churn_flag, ROUND(AVG(usage_count),2) AS Avg_usage
```

```
          FROM df GROUP BY Churn_flag'''
result()
```

Churn_flag boolean	Avg_usage double
true	49.04
false	50.33

```
#16. Customers Having Low Satisfaction.

query= '''SELECT Account_id, Subscription_id, Avg_satisfaction FROM df
          WHERE Avg_satisfaction < 1 ORDER BY Avg_satisfaction'''
result()
```

Account_id varchar	Subscription_id varchar	Avg_satisfaction double
A-e6afc1	S-98dd7d	0.0
A-e6afc1	S-de473d	0.0
A-e6afc1	S-e66964	0.0
A-e6afc1	S-02b3a1	0.0
A-91e948	S-61a87e	0.0
A-91e948	S-e2e052	0.0
A-91e948	S-7fb77e	0.0
A-e6afc1	S-111683	0.0
A-91e948	S-c14b9d	0.0
A-794a0b	S-973625	0.0
.	.	.
.	.	.
.	.	.
A-039727	S-9de4eb	0.0
A-039727	S-a6285f	0.0
A-039727	S-a575f5	0.0
A-d535f6	S-9a410c	0.0
A-d535f6	S-68b261	0.0
A-039727	S-7079c2	0.0

A-039727	S-864157	0.0
A-039727	S-0ec903	0.0
A-039727	S-e6a17a	0.0
A-ffdfd5	S-ef7534	0.0
57 rows (20 shown)		3 columns

#17. Find Customers on Annual Billing.

```
query= '''SELECT Account_id, Subscription_id, Billing_frequency FROM df
          WHERE Billing_frequency = 'Annual' '''
result()
```

Account_id varchar	Subscription_id varchar	Billing_frequency varchar
A-779e4e	S-92f228	Annual
A-779e4e	S-18356d	Annual
A-779e4e	S-11066a	Annual
A-f17767	S-7de656	Annual
A-d792a6	S-78c894	Annual
A-977ca0	S-ea5984	Annual
A-484819	S-18f80d	Annual
A-39b19f	S-0e4b0c	Annual
A-ed510f	S-709b46	Annual
A-a7bd4b	S-9a8f71	Annual
.	.	.
.	.	.
.	.	.
A-592832	S-0c63de	Annual
A-ced9aa	S-a72188	Annual
A-5b1bcd	S-e9257d	Annual
A-ff79f2	S-4f0027	Annual
A-82d8a6	S-9d7c19	Annual
A-17939a	S-8707e7	Annual
A-463db0	S-3eb56d	Annual
A-524364	S-7480e6	Annual
A-0f7d77	S-47048b	Annual

A-0f6450	S-9c9700	Annual

2087 rows (20 shown) 3 columns

#18. Top Countries with Highest Churn.

```
query= '''SELECT distinct Country, COUNT(*) AS Churned_customers FROM df
          WHERE Churn_flag = 'true' GROUP BY Country
          order by Churned_customers desc'''
result()
```

Country varchar	Churned_customers int64
US	242
UK	56
AU	31
IN	25
FR	20
DE	18
CA	16

#19. Plan-wise Churn Rate.

```
query= '''SELECT Plan_tier_after, COUNT(*) AS Total_customers,
          SUM(CASE WHEN Churn_flag = 'true' THEN 1 ELSE 0 END)
          AS Churned_customers FROM df GROUP BY Plan_tier_after'''
result()
```

Plan_tier_after varchar	Total_customers int64	Churned_customers int128
Basic	1356	128
Pro	1416	134

Enterprise	1450	146

#20. Average Revenue Per Seat.

```
query= '''SELECT ROUND(AVG(Arr_amount / Seats), 2) AS Avg_revenue_per_seat FROM df'''
result()
```

Avg_revenue_per_seat double
1333.02

#21. Industry with Highest Revenue.

```
query= '''SELECT Industry, SUM(Arr_amount) AS Total_revenue FROM df
          GROUP BY Industry ORDER BY total_revenue DESC'''
result()
```

Industry varchar	Total_revenue int128
FinTech	32033952
DevTools	28843848
Cybersecurity	27039816
EdTech	24518004
HealthTech	23629344

#22. Countries with Highest Satisfaction.

```
query= '''SELECT Country, ROUND(AVG(Avg_satisfaction),3) AS Avg_satisfaction FROM df
```

```
          GROUP BY Country ORDER BY Avg_satisfaction DESC'''
result()
```

Country varchar	Avg_satisfaction double
IN	4.007
FR	3.962
DE	3.96
UK	3.943
US	3.933
CA	3.881
AU	3.65

```
#23. Trial Customers Converted to Paid.

query= '''SELECT COUNT(*) AS Converted_customers FROM df
          WHERE Is_trial = 1 AND Churn_flag = 0'''
result()
```

Converted_customers int64
768

```
#24. Plan Upgrade Analysis.

query= '''SELECT Plan_tier_before, Plan_tier_after, COUNT(*) AS total_upgrades
          FROM df WHERE Upgrade_flag = 1 GROUP BY Plan_tier_before,
          Plan_tier_after ORDER BY Total_upgrades DESC'''
result()
```

Plan_tier_before	Plan_tier_after	total_upgrades

varchar	varchar	int64
Pro	Enterprise	537
Basic	Pro	510
Basic	Enterprise	492

#25. Plan Downgrade Analysis.

```
query= '''SELECT Plan_tier_before, Plan_tier_after, COUNT(*) AS Total_downgrades
          FROM df WHERE Downgrade_flag = 1 GROUP BY Plan_tier_before,
          Plan_tier_after ORDER BY Total_downgrades DESC'''
result()
```

Plan_tier_before varchar	Plan_tier_after varchar	Total_downgrades int64
Pro	Basic	456
Enterprise	Basic	437
Enterprise	Pro	413

#26. Customers with Longest Subscription Duration.

```
query= '''SELECT Account_id, Subscription_id, DATEDIFF('day',
          STRPTIME(Start_date, '%m/%d/%Y'), STRPTIME(End_date, '%m/%d/%Y'))
          AS Duration_days FROM df ORDER BY Duration_days DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Duration_days int64
A-779e4e	S-92f228	722
A-af0cd2	S-50aaaa	719
A-779e4e	S-18356d	703
A-7df7a7	S-6676fa	695

A-d792a6	S-47ce2d	694
A-d792a6	S-78c894	683
A-977ca0	S-ea5984	680
A-39b19f	S-6904e6	677
A-484819	S-18f80d	677
A-ed510f	S-709b46	663
10 rows		3 columns

#27. Churned Customers with Highest Revenue Loss.

```
query= '''SELECT Account_id, Subscription_id,Mrr_amount, Arr_amount FROM df
          WHERE Churn_flag = 1 ORDER BY Arr_amount DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Mrr_amount int64	Arr_amount int64
A-118f1c	S-9f8b23	25472	305664
A-d4e0d4	S-7904d3	23283	279396
A-ce550d	S-79d1e0	21691	260292
A-a6d261	S-50c6f5	20497	245964
A-c37cab	S-f4a4c8	19502	234024
A-66224b	S-021985	18308	219696
A-5c9849	S-6bd30a	18109	217308
A-eb7c38	S-615915	17910	214920
A-f44180	S-7da02b	17711	212532
A-5a215a	S-e04d93	17313	207756
10 rows			4 columns

#28. Average Usage by Plan Tier.

```
query= '''SELECT Plan_tier_after, ROUND(AVG(Usage_count),3) AS Avg_usage
```

```
          FROM df GROUP BY Plan_tier_after ORDER BY Avg_usage DESC'''
result()
```

Plan_tier_after varchar	Avg_usage double
Pro	50.27
Enterprise	50.19
Basic	50.162

#29. Revenue Generated by Referral Source.

```
query= '''SELECT Referral_source, SUM(Arr_amount) AS Total_revenue FROM df
          GROUP BY Referral_source ORDER BY Total_revenue DESC'''
result()
```

Referral_source varchar	Total_revenue int128
Organic	33584484
Other	28532736
Ads	25759428
Partner	25130916
Event	23057400

#30. Most Common Billing Frequency per Country.

```
query= '''SELECT Country, Billing_frequency, COUNT(*) AS Frequency_count
          FROM df GROUP BY Country, Billing_frequency
          ORDER BY Frequency_count DESC'''
result()
```

Country	Billing_frequency	Frequency_count

varchar	varchar	int64
US	Monthly	1278
US	Annual	1235
UK	Monthly	250
UK	Annual	247
IN	Monthly	219
IN	Annual	205
AU	Annual	137
AU	Monthly	124
DE	Monthly	93
FR	Annual	93
CA	Monthly	91
CA	Annual	87
DE	Annual	83
FR	Monthly	80
14 rows		3 columns

#31. Countries with Highest Churn Revenue Loss.

```
query= '''SELECT Country, SUM(Arr_amount) AS Total_loss FROM df
          WHERE Churn_flag = 1 GROUP BY Country ORDER BY Total_loss DESC'''
result()
```

Country varchar	Total_loss int128
US	7748652
UK	2103348
FR	1206756
AU	1062588
IN	979068
DE	529332
CA	519924

```
#32. Satisfaction vs Churn.

query= '''SELECT Churn_flag, ROUND(AVG(Avg_satisfaction),3) AS Avg_satisfaction
          FROM df GROUP BY Churn_flag'''
result()
```

Churn_flag boolean	Avg_satisfaction double
false	3.921
true	3.953

```
#33. Ticket Count vs Satisfaction.

query= '''SELECT Total_tickets, ROUND(AVG(Avg_satisfaction),3) AS Avg_satisfaction
          FROM df GROUP BY Total_tickets ORDER BY Total_tickets DESC'''
result()
```

Total_tickets int64	Avg_satisfaction double
11	4.1
10	3.8
9	3.994
8	3.986
7	3.943
6	4.021
5	4.027
4	3.977
3	4.013
2	3.928
1	3.786
0	0.0
12 rows	2 columns

#34. High Revenue Customers with Low Satisfaction.

```
query= '''SELECT Account_id, Subscription_id, Mrr_amount, Avg_satisfaction
          FROM df WHERE Avg_satisfaction < 1 ORDER BY Mrr_amount DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Mrr_amount int64	Avg_satisfaction double
A-e6afc1	S-98dd7d	10149	0.0
A-91e948	S-683e1e	8159	0.0
A-91e948	S-92ae0a	8159	0.0
A-794a0b	S-38ac0c	6169	0.0
A-ffdfd5	S-0a5cd7	5970	0.0
A-039727	S-a6285f	5970	0.0
A-039727	S-9de4eb	4975	0.0
A-039727	S-ed55d2	4975	0.0
A-039727	S-574b52	4975	0.0
A-039727	S-7079c2	4975	0.0
10 rows			4 columns

#35. Low Revenue Customers with High Satisfaction.

```
query= '''SELECT Account_id, Subscription_id, Mrr_amount, Avg_satisfaction
          FROM df WHERE Avg_satisfaction > 4 ORDER BY Mrr_amount LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Mrr_amount int64	Avg_satisfaction double
A-2b9d3e	S-8d4b81	19	4.3
A-e19ff6	S-4047a8	19	4.7

A-5a184f	S-c2b60b	19	4.3
A-9289f6	S-e77d56	19	4.2
A-fd7ad3	S-d19797	19	4.1
A-9077b0	S-5f30ce	38	4.3
A-f1f639	S-ab212b	38	4.6
A-13466a	S-2f8dbb	38	4.3
A-6f8ad2	S-aa20eb	49	4.2
A-e19ff6	S-367dee	49	4.7
10 rows			4 columns

#36. Auto Renewal Impact on Revenue.

```
query= '''SELECT Auto_renewal, ROUND(SUM(Mrr_amount),3) AS Total_mrr FROM df
          GROUP BY Auto_renewal'''
result()
```

Auto_renewal boolean	Total_mrr int128
true	9104308
false	2234439

#37. Average Errors by Plan Tier.

```
query= '''SELECT Plan_tier_after, ROUND(AVG(Error_count),3) AS Avg_errors
          FROM df GROUP BY Plan_tier_after ORDER BY Avg_errors DESC'''
result()
```

Plan_tier_after varchar	Avg_errors double
Pro	2.886
Enterprise	2.803

Basic	2.76

#38. Most Profitable Industries.

```
query= '''SELECT Industry, SUM(Arr_amount) AS Total_revenue FROM df
          GROUP BY Industry ORDER BY Total_revenue DESC LIMIT 1'''
result()
```

Industry varchar	Total_revenue int128
FinTech	32033952

#39. Customers Using Product Most Efficiently.

```
query= '''SELECT Account_id, Subscription_id, Usage_count FROM df
          ORDER BY Usage_count DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Usage_count int64
A-781cc0	S-0896f4	148
A-7d514c	S-8a46fe	143
A-3b37d9	S-02b754	138
A-726cfa	S-9ce22d	136
A-0aaf6e	S-52385a	134
A-e89151	S-7e8699	132
A-5247b3	S-4f7b91	131
A-9f0b68	S-115823	130
A-fdfc91	S-d933e7	129
A-427b69	S-33deb8	127
10 rows		3 columns

```
#40. Customers with Frequent Errors and High Tickets.

query= '''SELECT Account_id, Subscription_id, Error_count, Total_tickets
          FROM df WHERE Error_count > 7 AND Total_tickets > 7
          ORDER BY Error_count DESC, Total_tickets DESC'''
result()
```

Account_id varchar	Subscription_id varchar	Error_count int64	Total_tickets int64
A-dcc3f1	S-4d9635	10	9
A-0532a9	S-29597a	10	8
A-86902e	S-b83d8d	9	9
A-f446b6	S-54a3ed	9	8
A-ed510f	S-1ad0ca	8	9
A-c42f1f	S-1bf267	8	9
A-0532a9	S-951c06	8	8
A-0f6450	S-e11cde	8	8

```
#41. Average Subscription Duration by Industry.

query= '''SELECT Industry, ROUND(AVG(DATEDIFF('day', STRPTIME(Start_date, '%m/%d/%Y'),
          STRPTIME(End_date, '%m/%d/%Y'))),3) AS Avg_duration_days
          FROM df GROUP BY Industry ORDER BY Avg_duration_days DESC'''
result()
```

Industry varchar	Avg_duration_days double
HealthTech	171.58
DevTools	167.751
FinTech	163.711
Cybersecurity	148.344

EdTech	146.252

#42. Highest Paying Customers by Seat Count.

```
query= '''SELECT Account_id, Subscription_id, Seats, Mrr_amount
          FROM df ORDER BY Seats DESC, Mrr_amount DESC LIMIT 10'''
result()
```

Account_id varchar	Subscription_id varchar	Seats int64	Mrr_amount int64
A-56962b	S-e98121	163	32437
A-56962b	S-86e45a	163	7987
A-56962b	S-c7190f	163	7987
A-56962b	S-c8fa77	163	3097
A-56962b	S-4970c1	163	3097
A-18793f	S-155b3e	148	29452
A-18793f	S-198dbe	148	7252
A-18793f	S-ad8908	148	7252
A-18793f	S-74f146	148	7252
A-18793f	S-e854de	148	7252
10 rows			4 columns

#43. Churned Customers Who Downgraded.

```
query= '''SELECT Account_id, Subscription_id, Downgrade_flag
          FROM df WHERE Churn_flag = 1 AND Downgrade_flag = 1
          ORDER BY Downgrade_flag DESC'''
result()
```

Account_id varchar	Subscription_id varchar	Downgrade_flag boolean

A-f17767	S-7de656	true
A-b11cd0	S-7a36ee	true
A-39b19f	S-0e4b0c	true
A-e1b9cd	S-302856	true
A-484819	S-b27c39	true
A-9affe6	S-e7f571	true
A-158f4e	S-f3ff07	true
A-32728c	S-8b349c	true
A-4a4c2d	S-e133a5	true
A-9289f6	S-ae20d8	true
.	.	.
.	.	.
.	.	.
A-a45270	S-49aa80	true
A-e98302	S-008f3c	true
A-e7beab	S-8ff318	true
A-e351e1	S-0509b8	true
A-443b7c	S-d1a6c6	true
A-89ee83	S-f5f5b3	true
A-854864	S-bbafad	true
A-d27f50	S-6f488b	true
A-0b0d6d	S-6ef81d	true
A-6da850	S-42aaf0	true

135 rows (20 shown) 3 columns

#45. Revenue Lost Due to Churn by Industry.

```
query= '''SELECT Industry, ROUND(SUM(Arr_amount),2) AS Revenue_lost FROM df
          WHERE Churn_flag = 1 GROUP BY Industry ORDER BY Revenue_lost DESC'''
result()
```

Industry varchar	Revenue_lost int128
Cybersecurity	3348744
FinTech	3041352
DevTools	2863332

```
| HealthTech     |       2511324 |
| EdTech         |       2384916 |
└────────────────┴───────────────┘
```

```
#45. Top Referral Sources for High Revenue Customers.

query= '''SELECT Referral_source, COUNT(*) AS Premium_customers FROM df
          WHERE Arr_amount > (SELECT AVG(Arr_amount)FROM df)
          GROUP BY Referral_source ORDER BY premium_customers DESC'''
result()
```

```
┌─────────────────┬──────────────────────┐
│ Referral_source │ Premium_customers    │
│     varchar     │        int64         │
├─────────────────┼──────────────────────┤
│ Organic         │                  334 │
│ Other           │                  247 │
│ Ads             │                  241 │
│ Event           │                  231 │
│ Partner         │                  229 │
└─────────────────┴──────────────────────┘
```

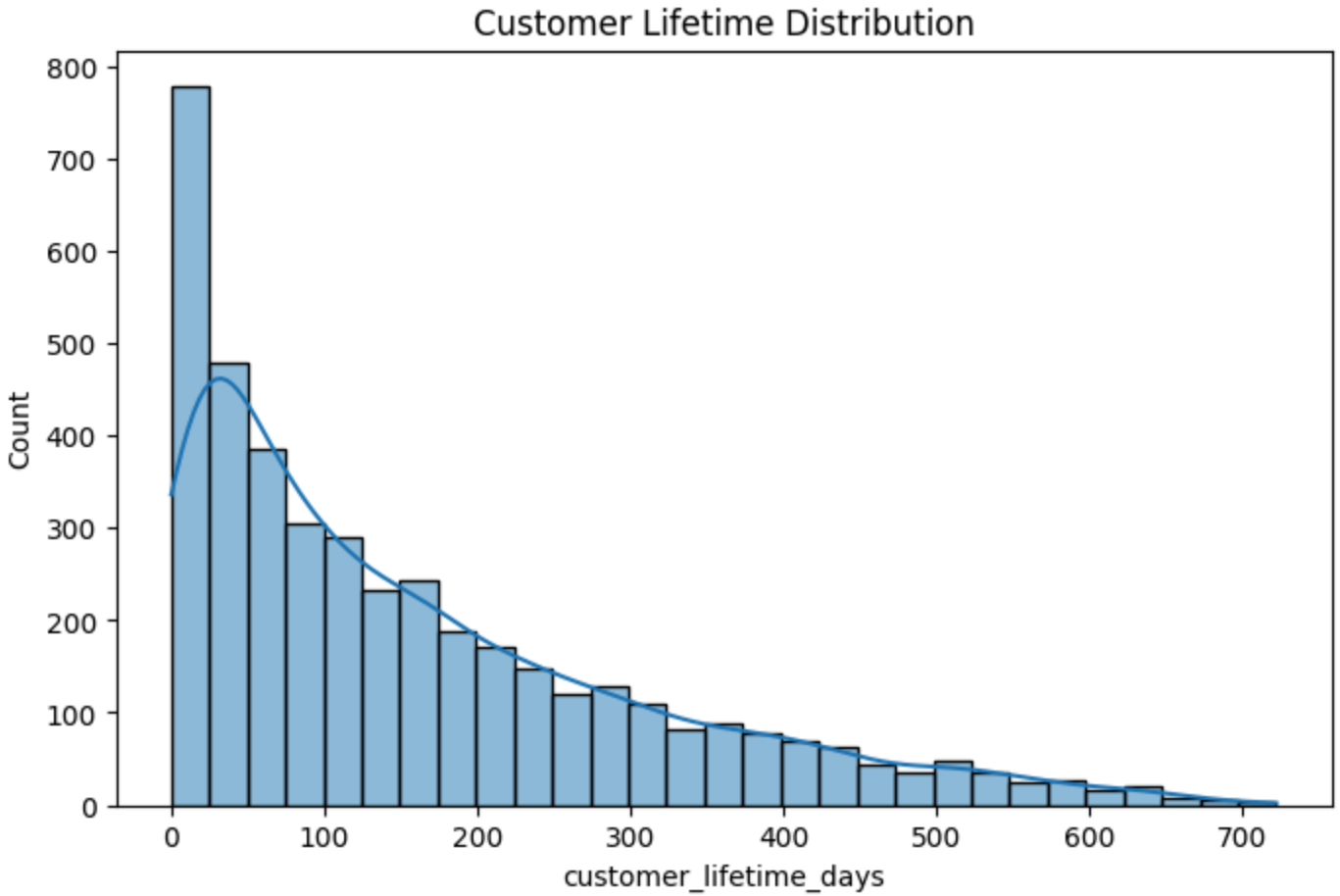
Conclusions from SQL Analysis

- Customer acquisition is influenced by different referral sources, with some channels contributing higher-value customers.
- Revenue distribution varies across industries and countries, indicating key profitable market segments.
- Certain subscription plans generate higher MRR and ARR, highlighting premium plan profitability.
- Trial users demonstrate different retention patterns, providing insights into trial-to-paid conversion effectiveness.
- Customers with plan upgrades indicate growth opportunities, while downgrades may signal dissatisfaction or reduced usage needs.
- Higher usage count and usage duration are associated with stronger engagement, whereas low engagement may indicate churn risk.

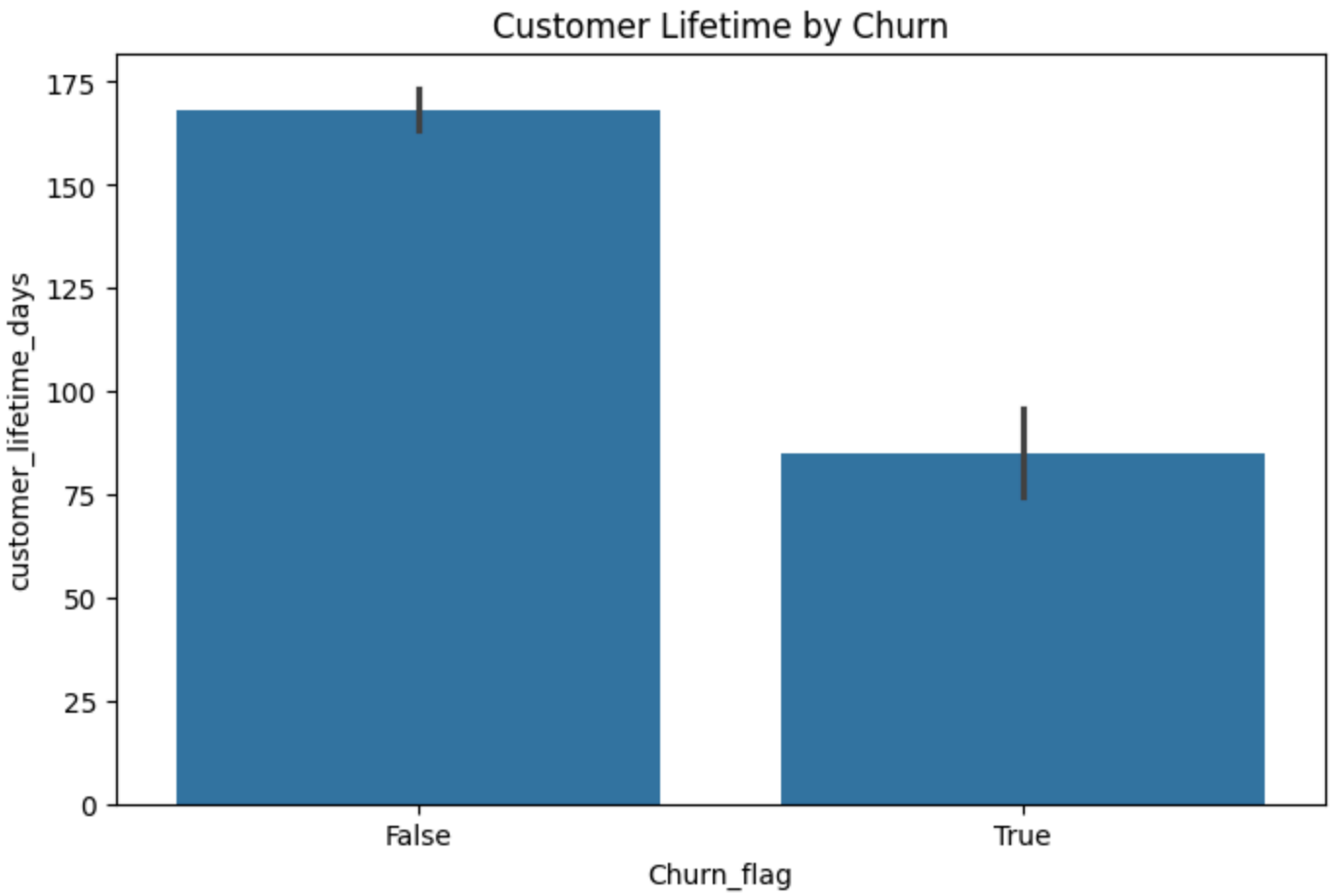
- Customers experiencing higher error counts and more support tickets may face product-related challenges, impacting satisfaction.
- Satisfaction levels vary across customer groups, suggesting that customer experience plays a major role in retention.
- Some industries and regions contribute disproportionately to revenue loss due to churn, helping identify high-risk business segments.
- Enterprise customers with higher seat counts contribute significantly to recurring revenue, making retention critical.

VI. Python EDA

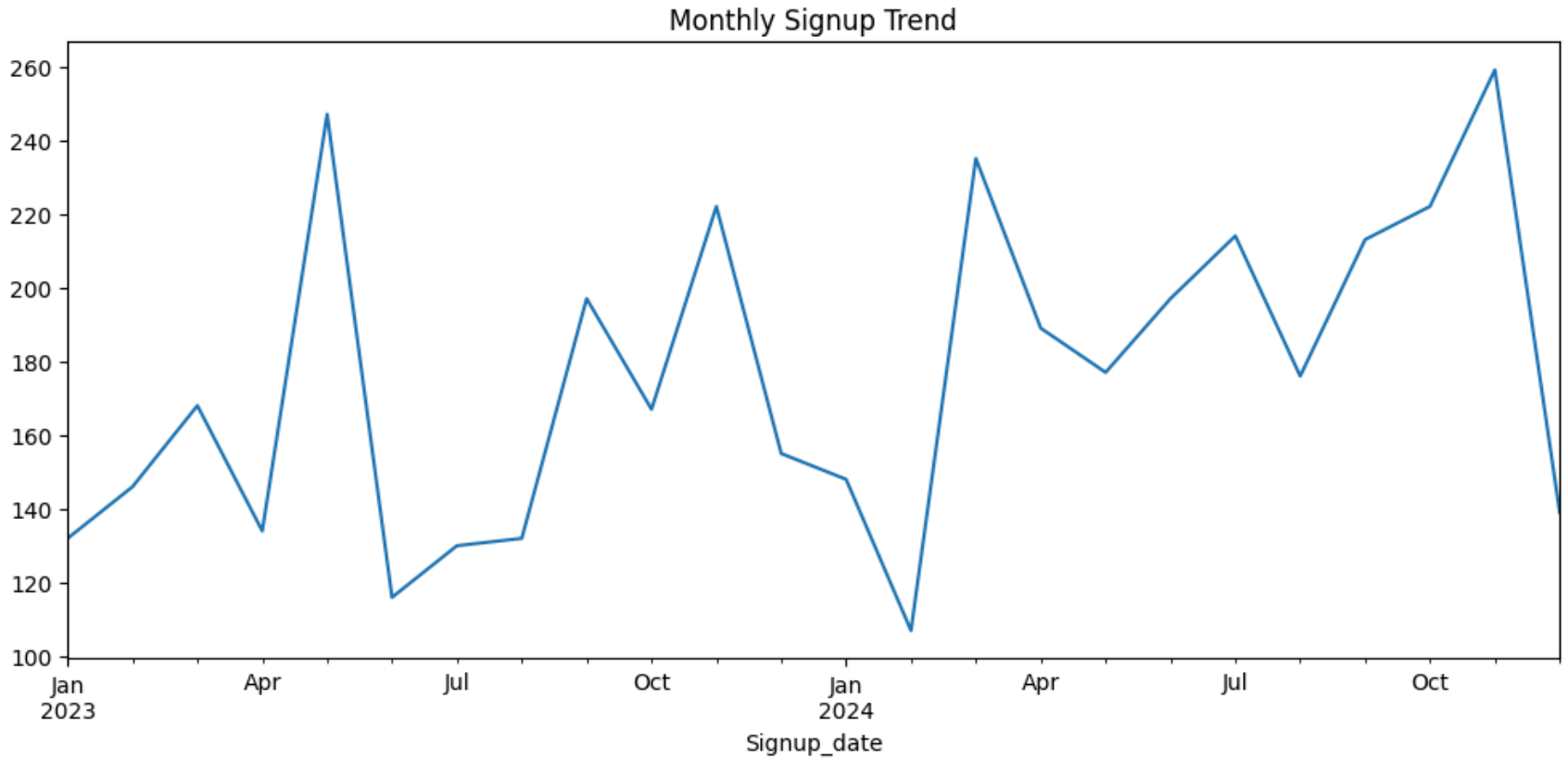
```
#1. How long do customers stay subscribed?

df['customer_lifetime_days'] = (pd.to_datetime(df['End_date']) -
    pd.to_datetime(df['Start_date'])).dt.days
plt.figure(figsize=(8,5))
sns.histplot( df['customer_lifetime_days'], kde=True)
plt.title("Customer Lifetime Distribution")
plt.show()
```



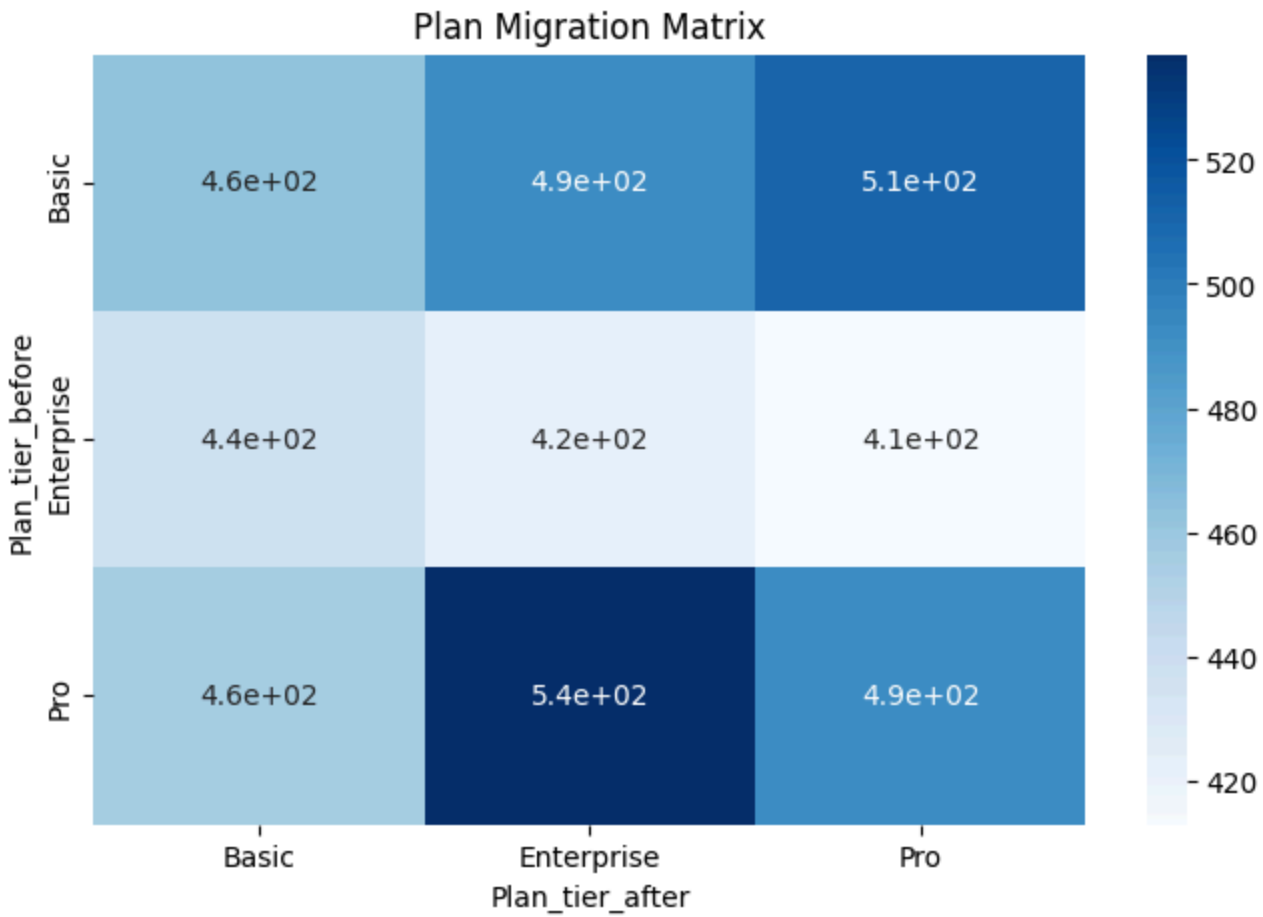

```
#2. Are churned customers leaving early?

plt.figure(figsize=(8,5))
sns.barplot(x='Churn_flag', y='customer_lifetime_days', data=df)
plt.title("Customer Lifetime by Churn")
plt.show()
```



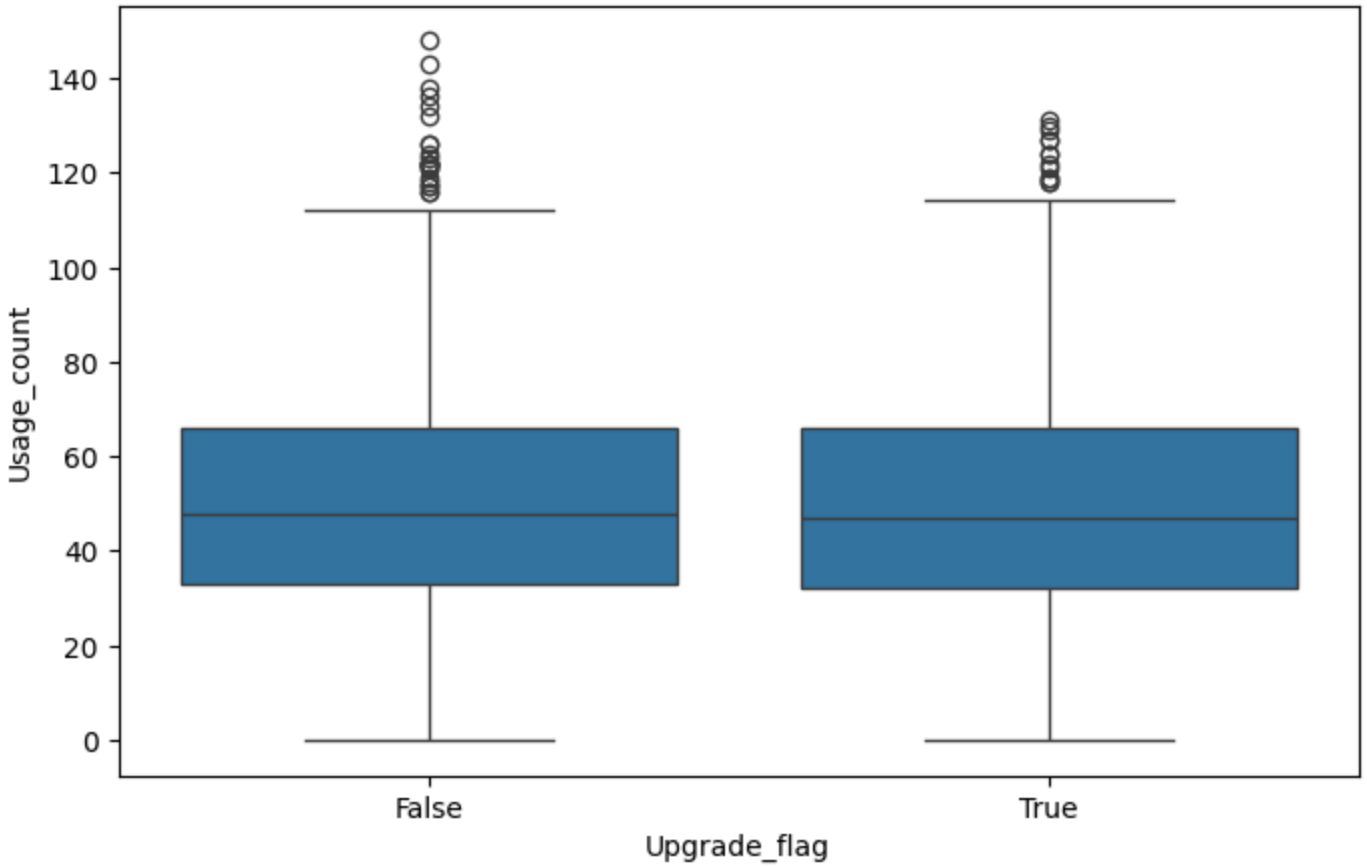

```
#3. Signup trend over time.

df['Signup_date'] = pd.to_datetime(df['Signup_date'])
monthly_signup = df.groupby(df['Signup_date'].dt.to_period('M')).size()
monthly_signup.plot(figsize=(12,5))
plt.title("Monthly Signup Trend")
plt.show()
```



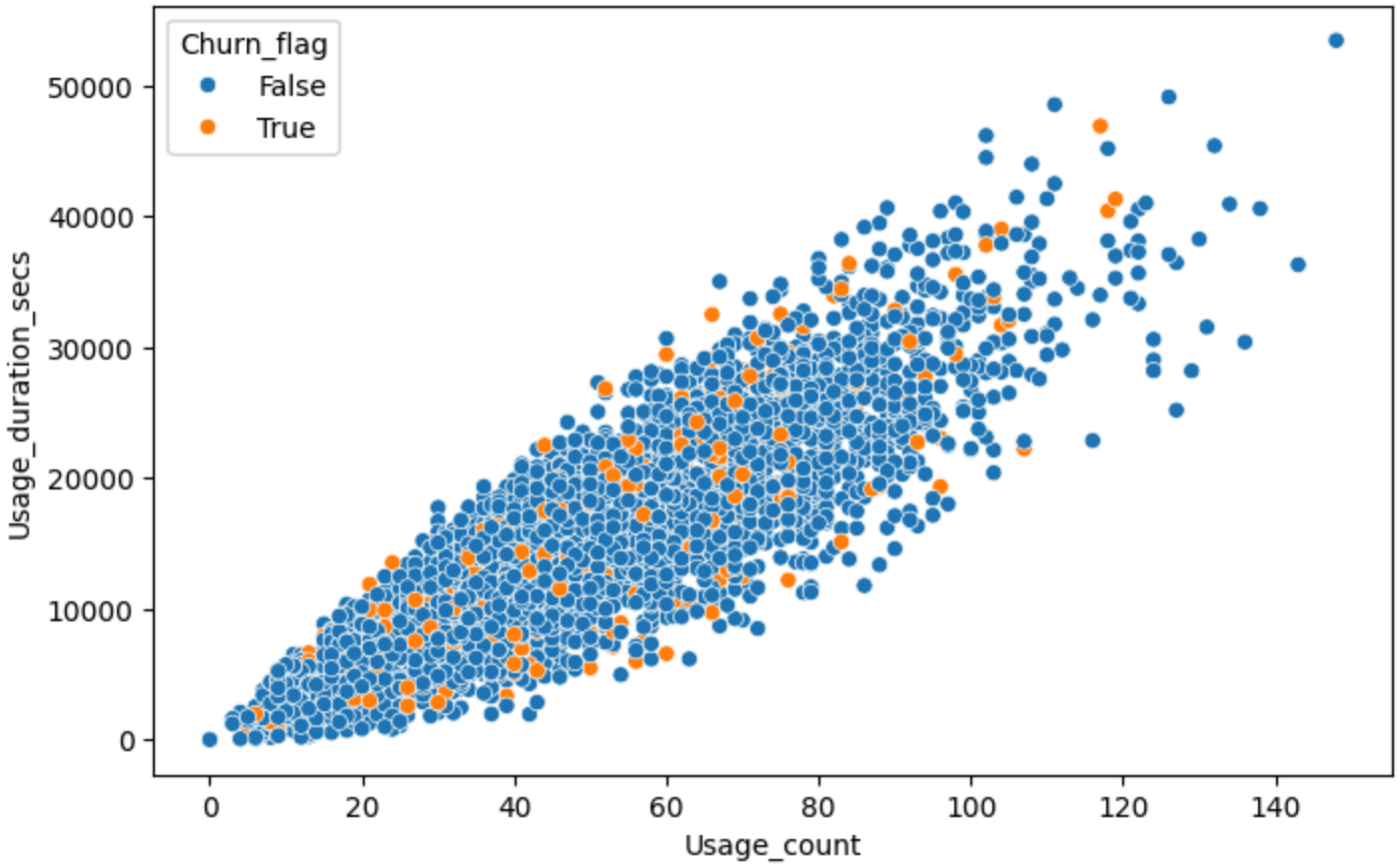

```
#4. Plan movement heatmap.

plan_transition = pd.crosstab(df['Plan_tier_before'],df['Plan_tier_after'])
plt.figure(figsize=(8,5))
sns.heatmap(plan_transition, annot=True,cmap='Blues')
plt.title("Plan Migration Matrix")
plt.show()
```



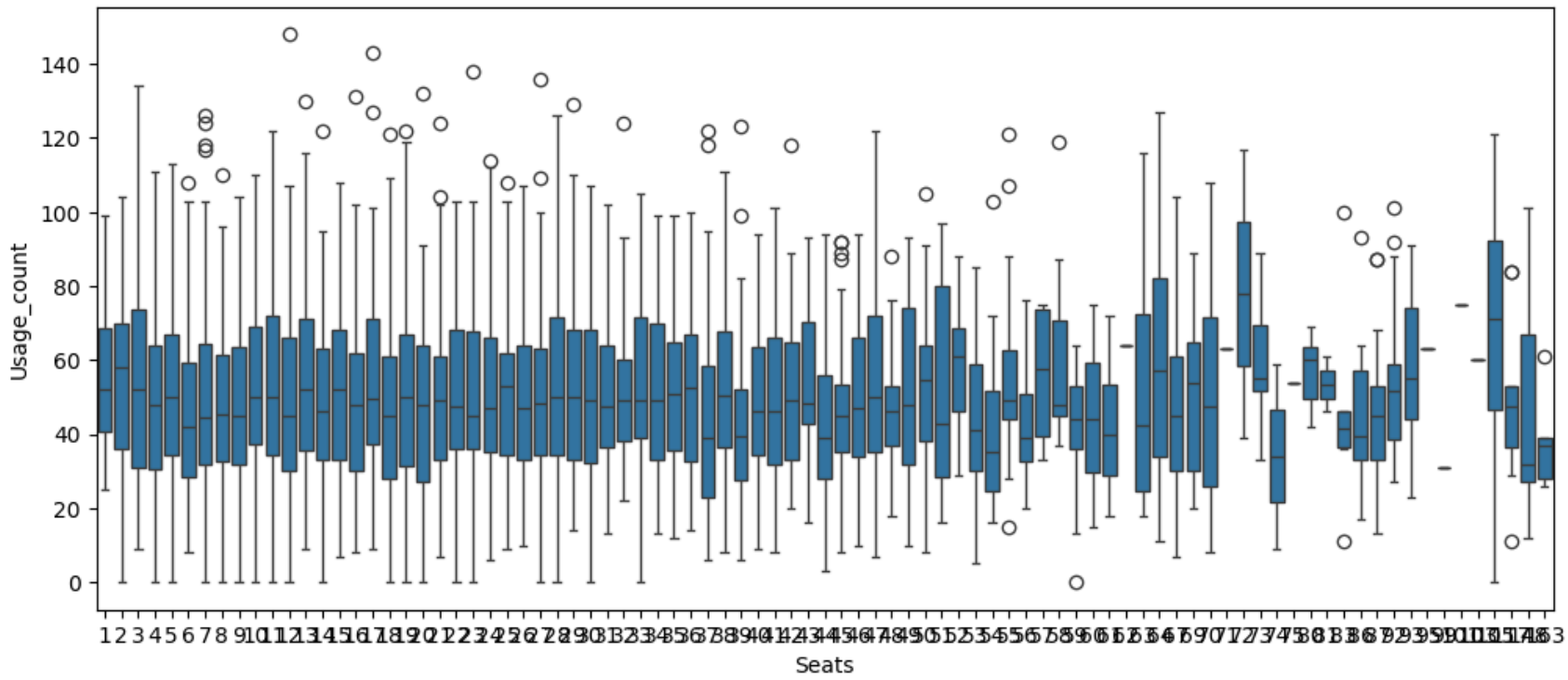

```
#5. Do upgraded customers use the platform more?

plt.figure(figsize=(8,5))
sns.boxplot(x='Upgrade_flag',y='Usage_count', data=df)
plt.show()
```



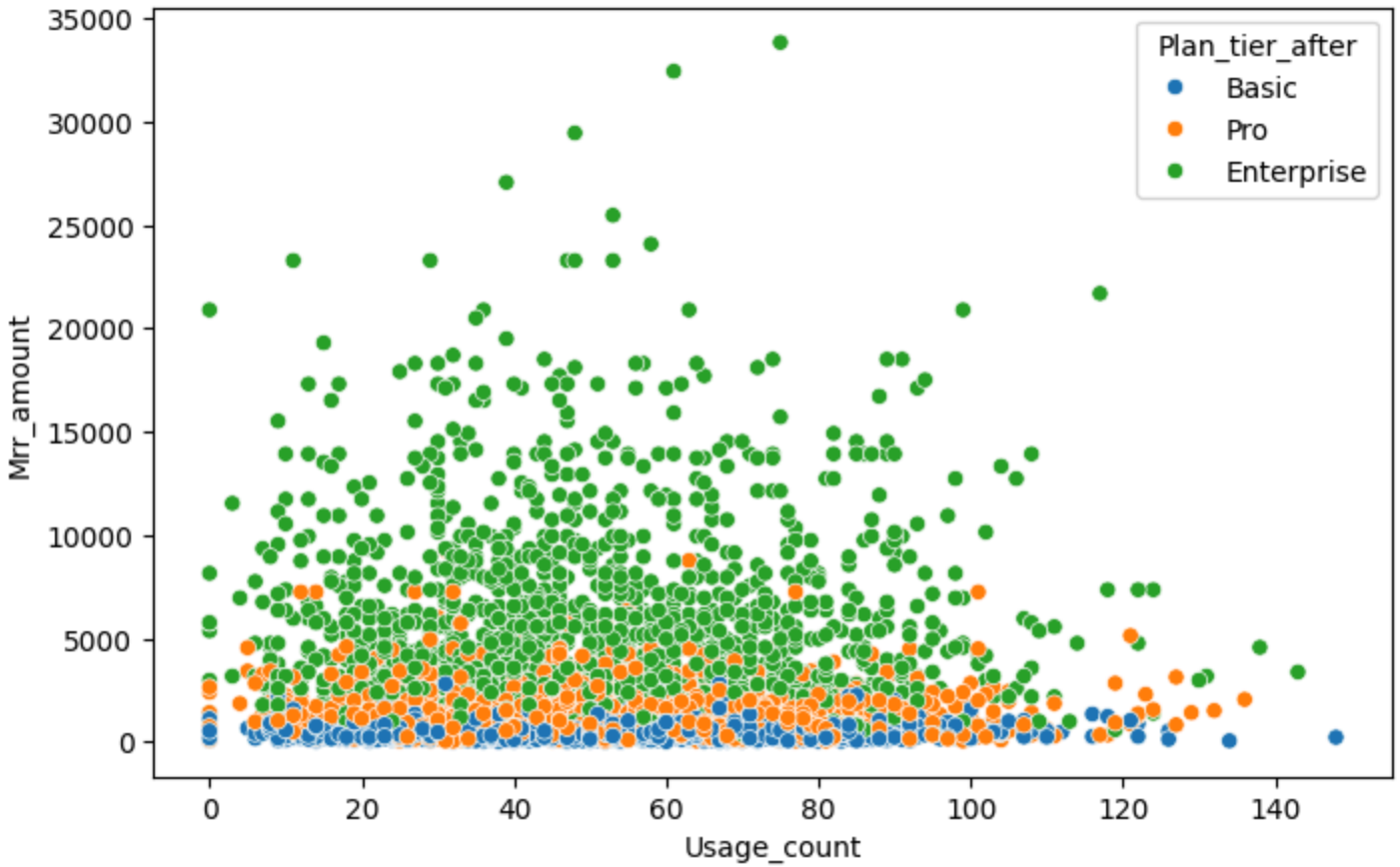

```
#6. Usage Count vs Duration Relationship.

plt.figure(figsize=(8,5))
sns.scatterplot(x='Usage_count', y='Usage_duration_secs', hue='Churn_flag',
                data=df)
plt.show()
```



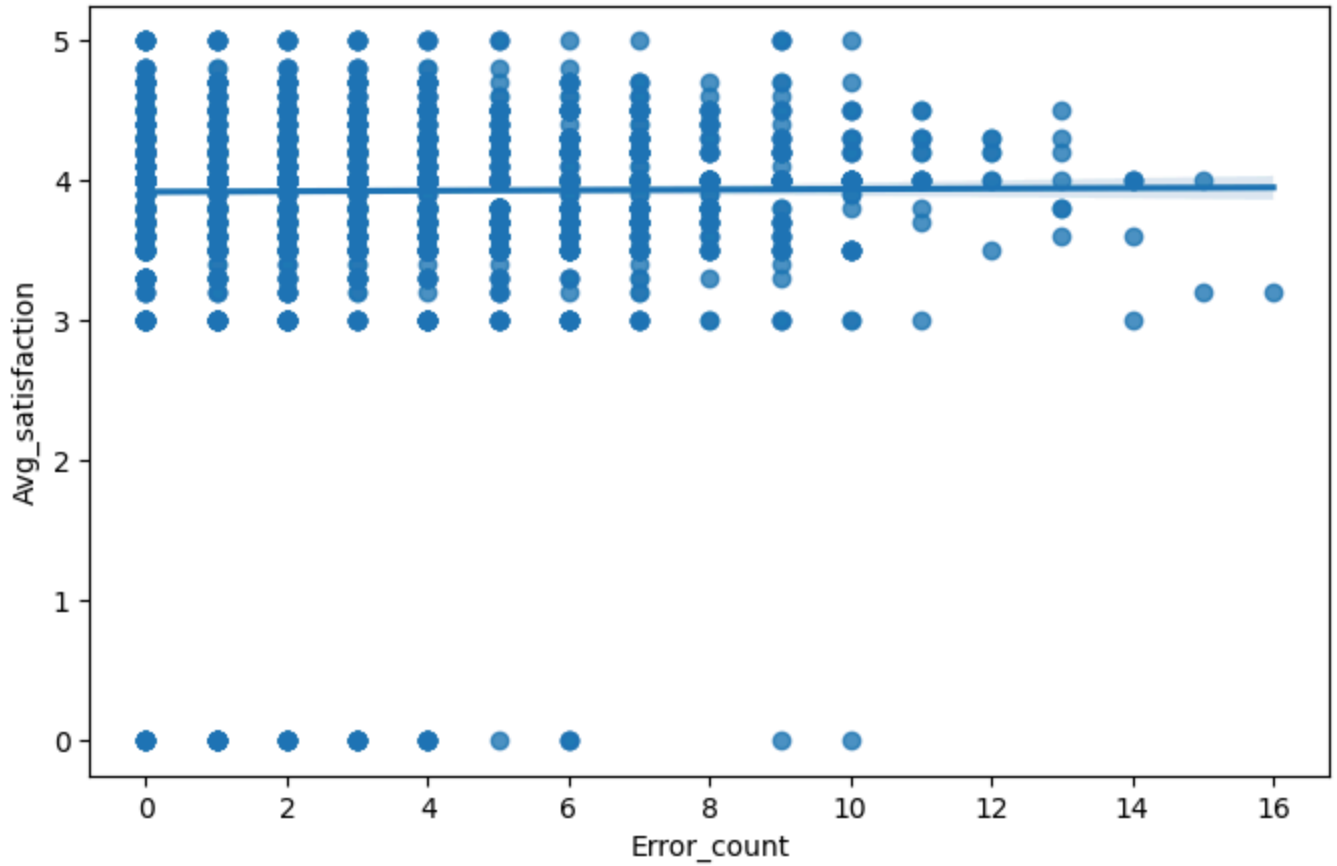

```
#7. Does team size affect engagement?

plt.figure(figsize=(12,5))
sns.boxplot(x='Seats', y='Usage_count', data=df)
plt.show()
```



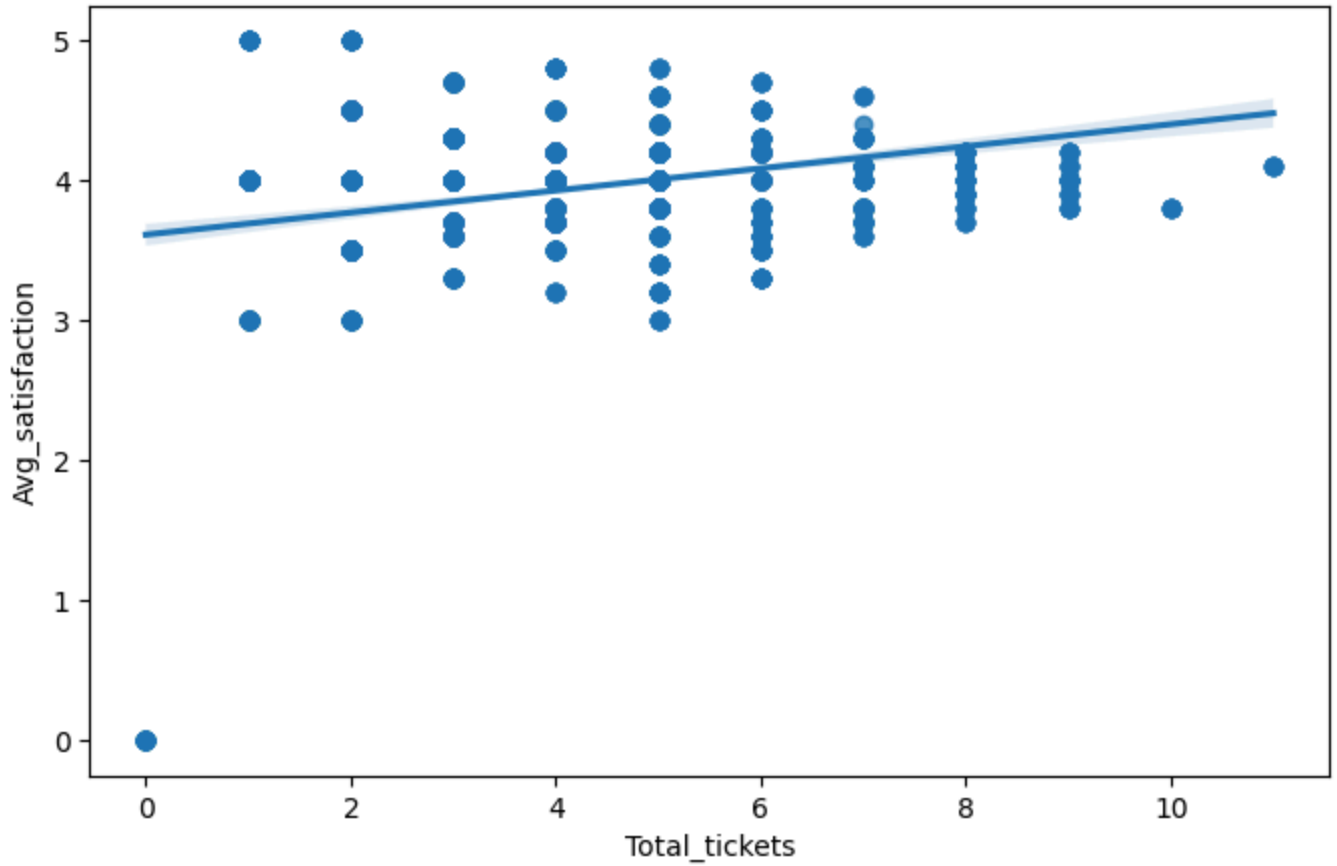

```
#8. Revenue vs Product Usage.

plt.figure(figsize=(8,5))
sns.scatterplot(x='Usage_count', y='Mrr_amount', hue='Plan_tier_after',
                data=df)
plt.show()
```



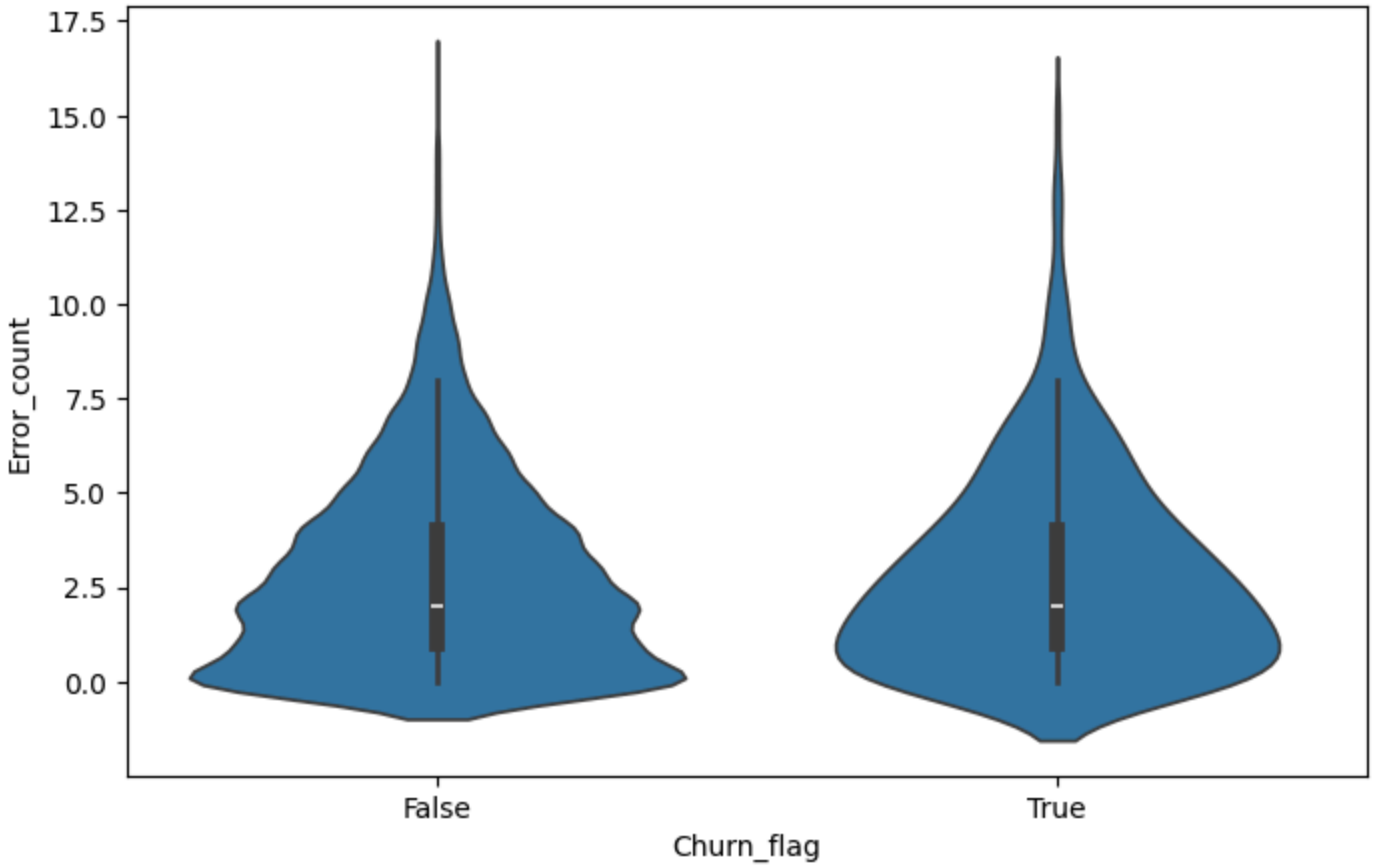

```
#9. Are customers with more errors less satisfied?

plt.figure(figsize=(8,5))
sns.regplot(x='Error_count', y='Avg_satisfaction', data=df)
plt.show()
```



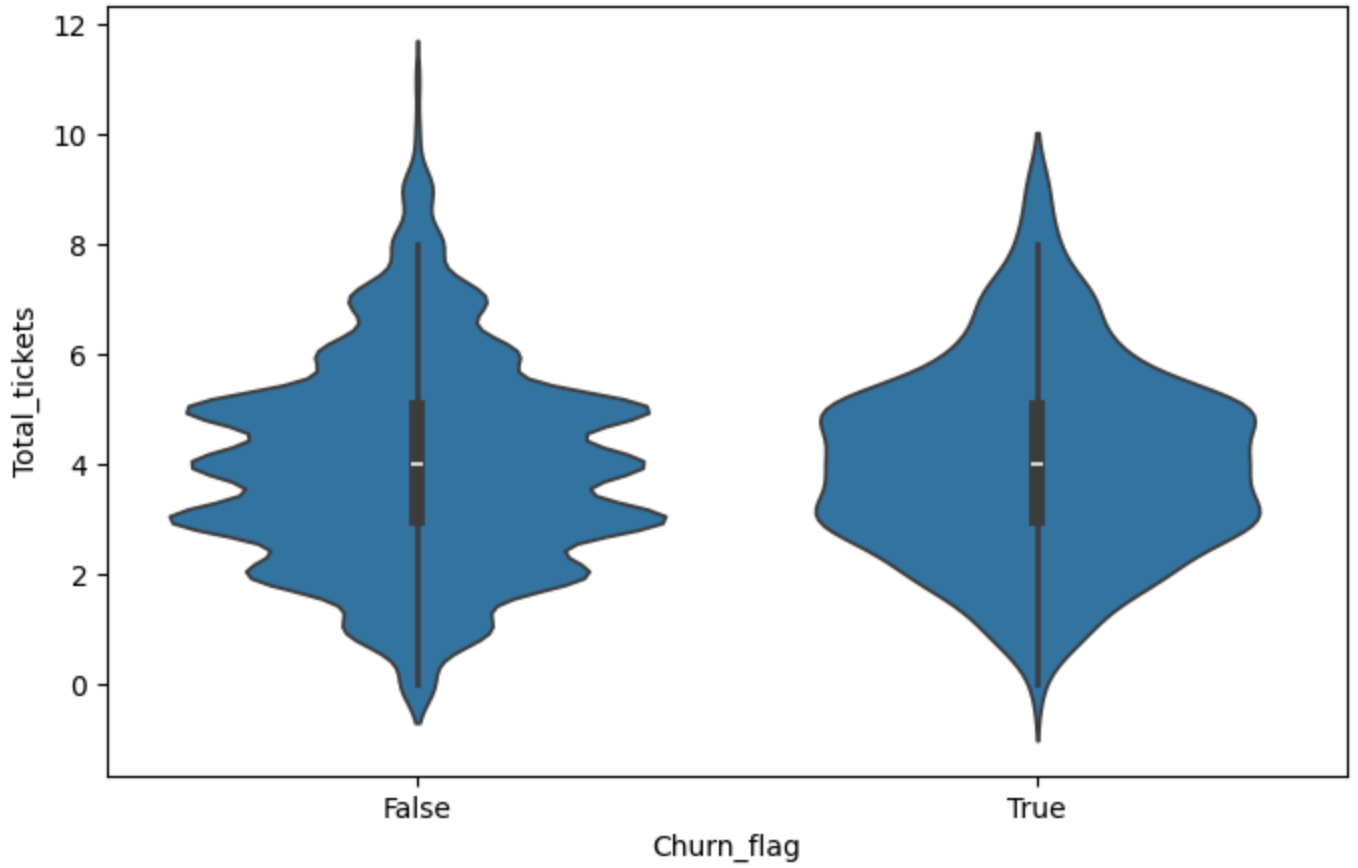

```
#10. Ticket load vs satisfaction.

plt.figure(figsize=(8,5))
sns.regplot(x='Total_tickets', y='Avg_satisfaction', data=df)
plt.show()
```



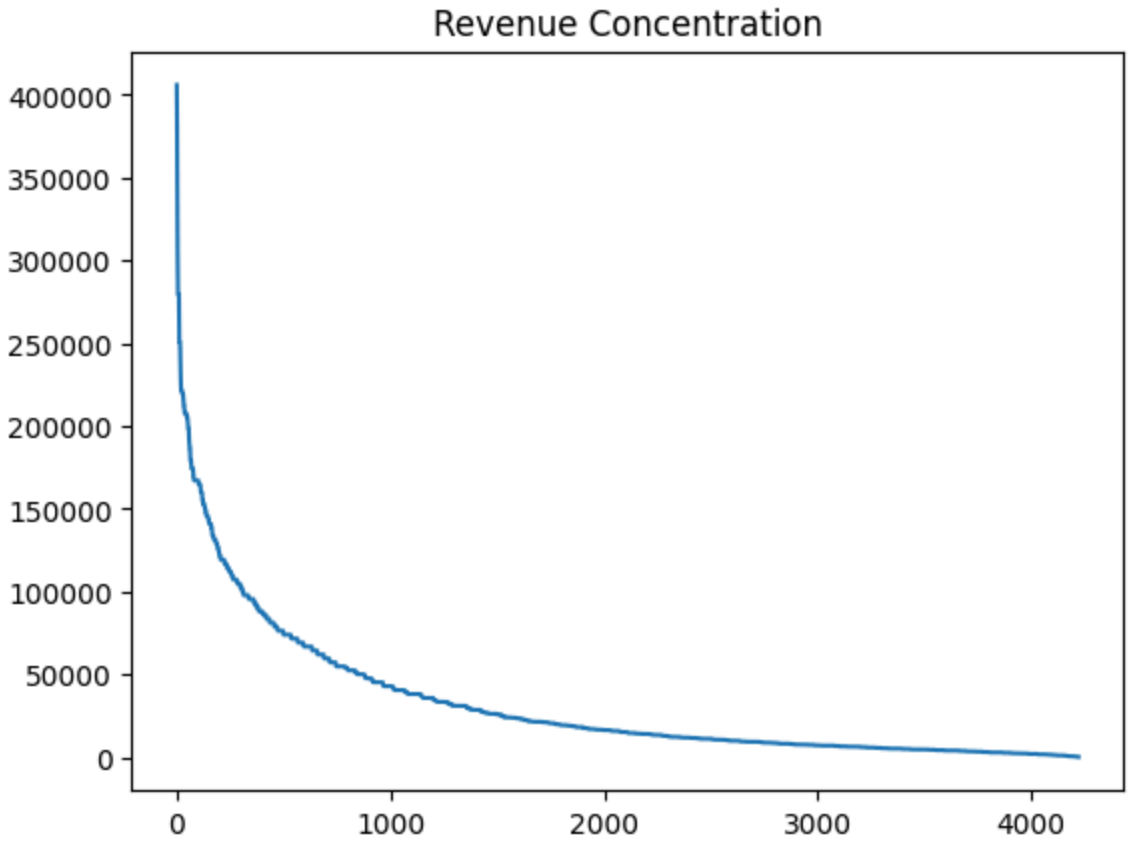

```
#11. Error count distribution by churn.

plt.figure(figsize=(8,5))
sns.violinplot(x='Churn_flag', y='Error_count', data=df)
plt.show()
```



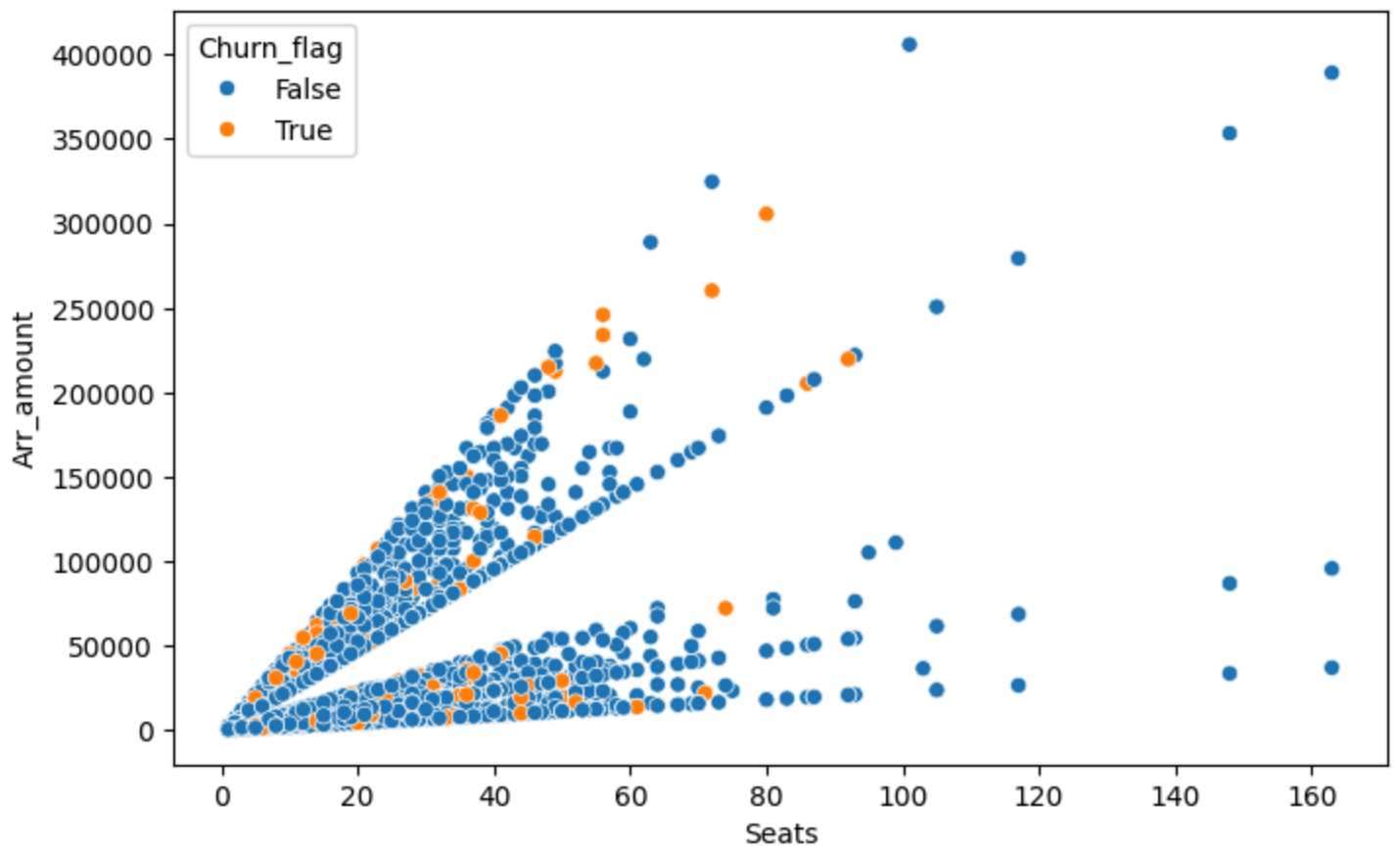

```
#12. Support burden by churn.

plt.figure(figsize=(8,5))
sns.violinplot(x='Churn_flag', y='Total_tickets', data=df)
plt.show()
```



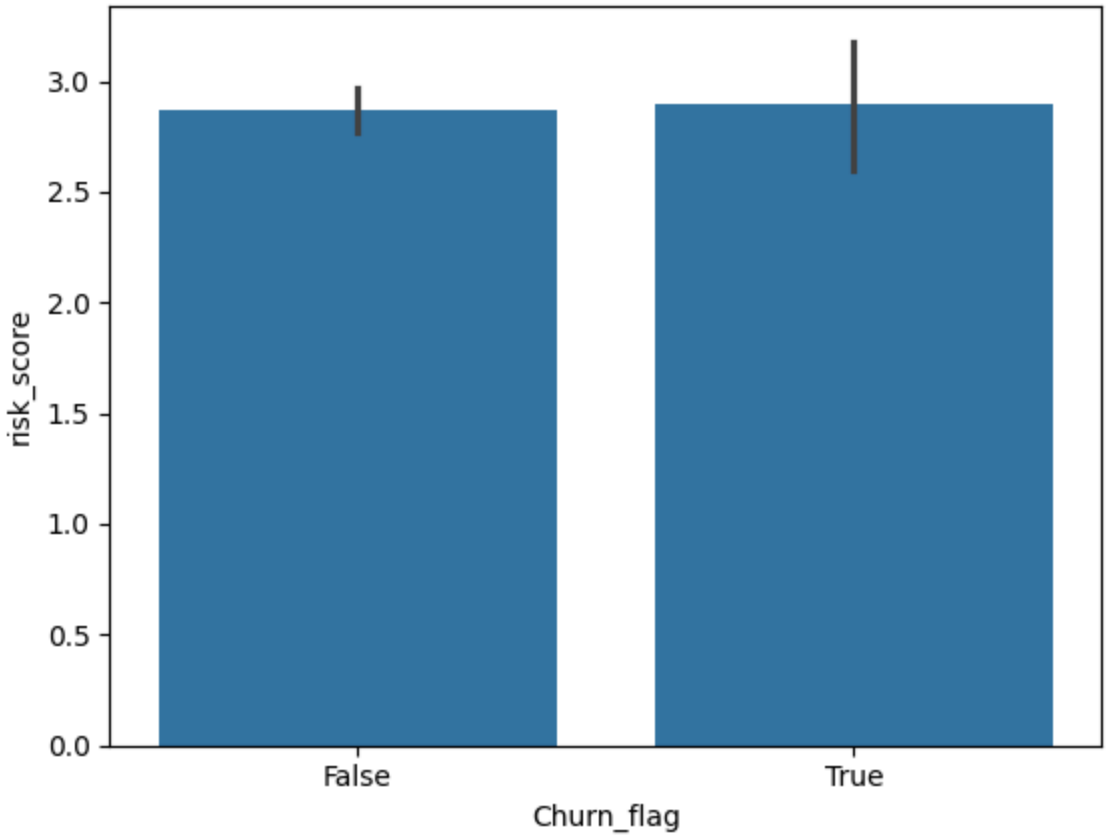

```
#13. Revenue Behavior Analysis.

revenue_sorted = df['Arr_amount'].sort_values(ascending=False)
revenue_sorted.reset_index(drop=True).plot()
plt.title("Revenue Concentration")
plt.show()
```



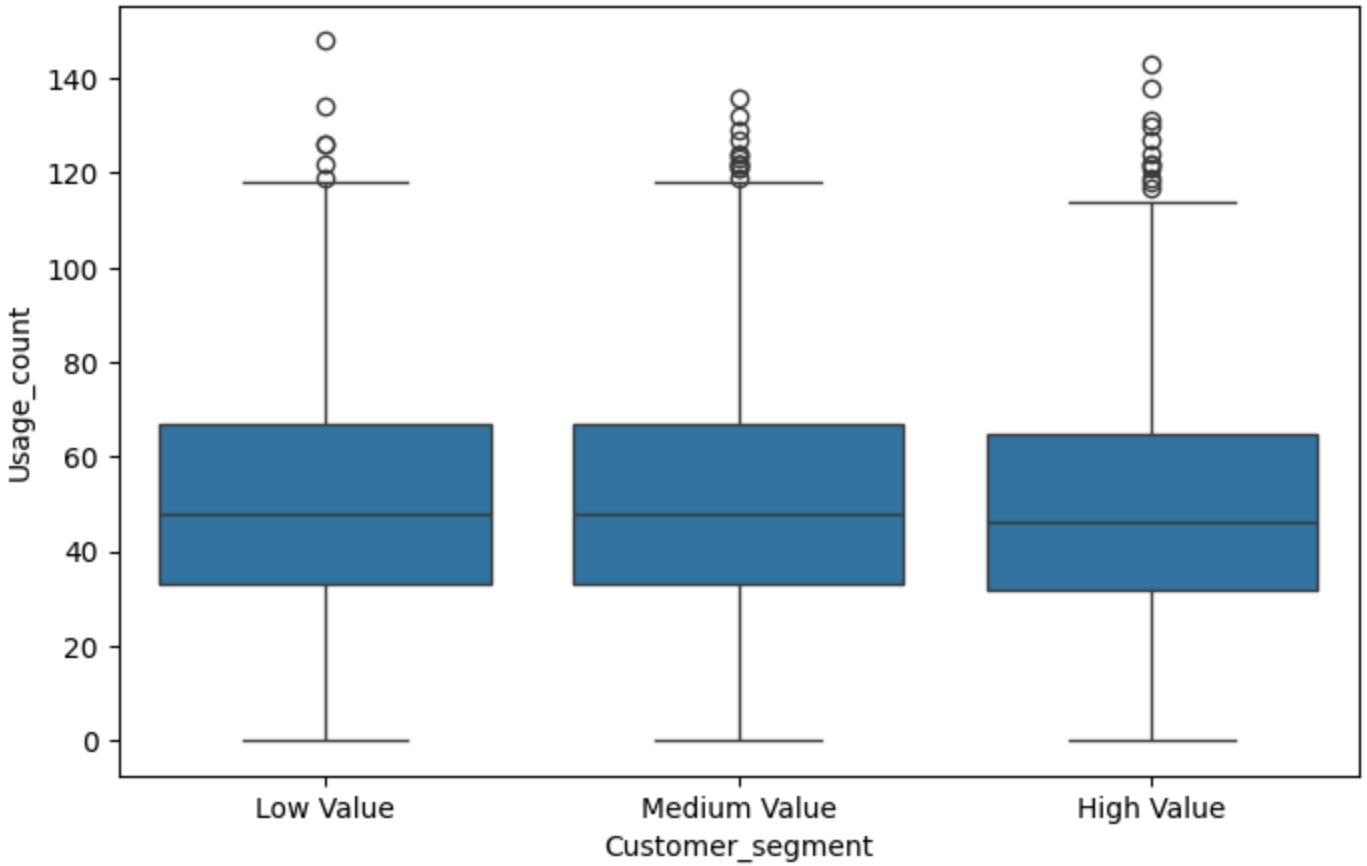

```
#14. Seats vs Revenue Relationship.

plt.figure(figsize=(8,5))
sns.scatterplot(x='Seats', y='Arr_amount', hue='Churn_flag', data=df)
plt.show()
```



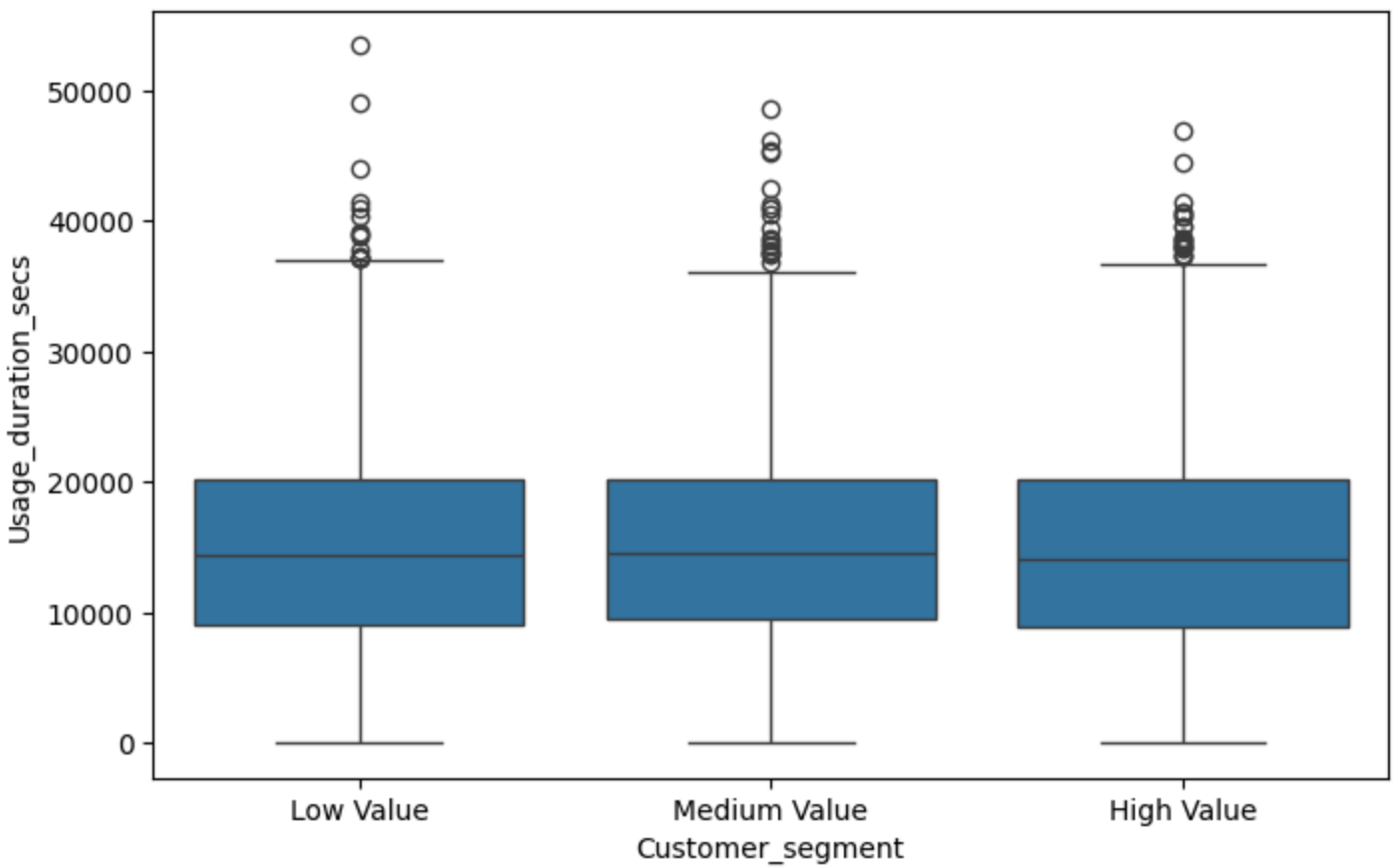

```
#15. Risk score distribution.

df['risk_score'] = (df['Error_count'] + df['Total_tickets'] - df['Avg_satisfaction'])
sns.barplot(x='Churn_flag', y='risk_score', data=df)
plt.show()
```



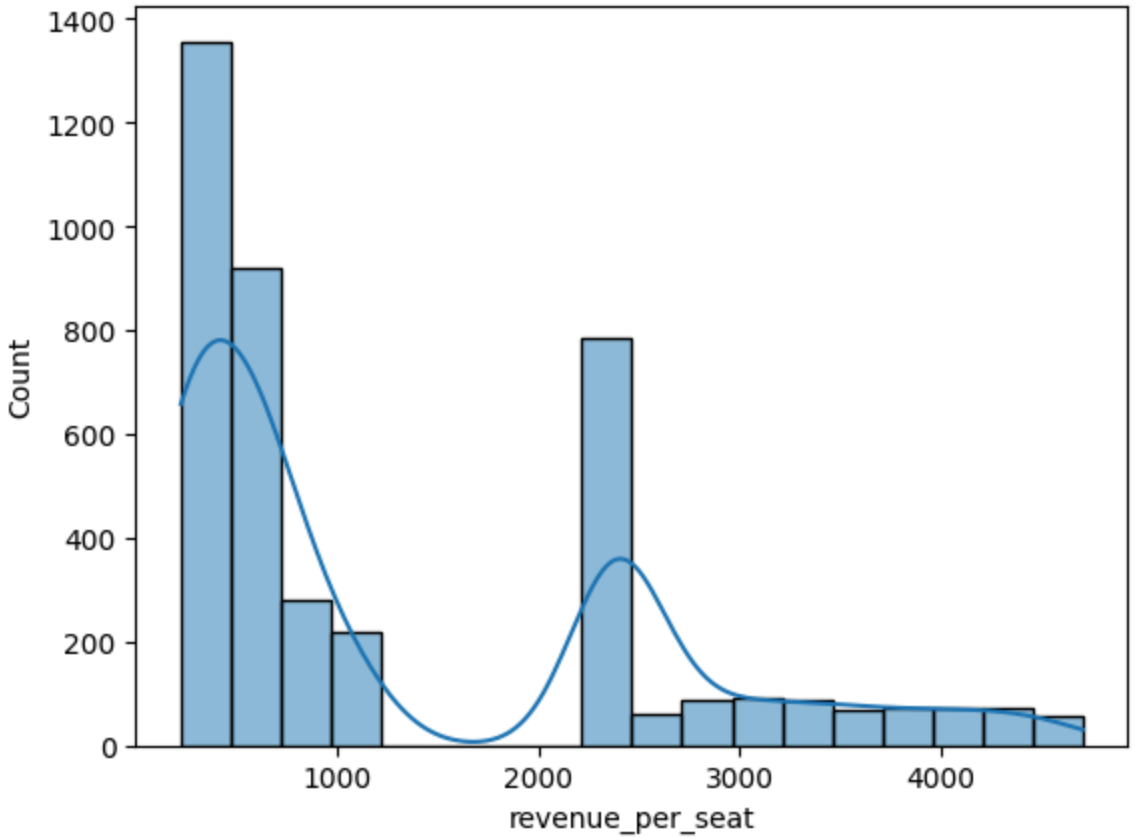

```
#16. Customer Segmentation EDA.

df['Customer_segment'] = pd.qcut(df['Arr_amount'],q=3,
                         labels=['Low Value', 'Medium Value', 'High Value'])
plt.figure(figsize=(8,5))
sns.boxplot(x='Customer_segment', y='Usage_count', data=df)
plt.show()
```



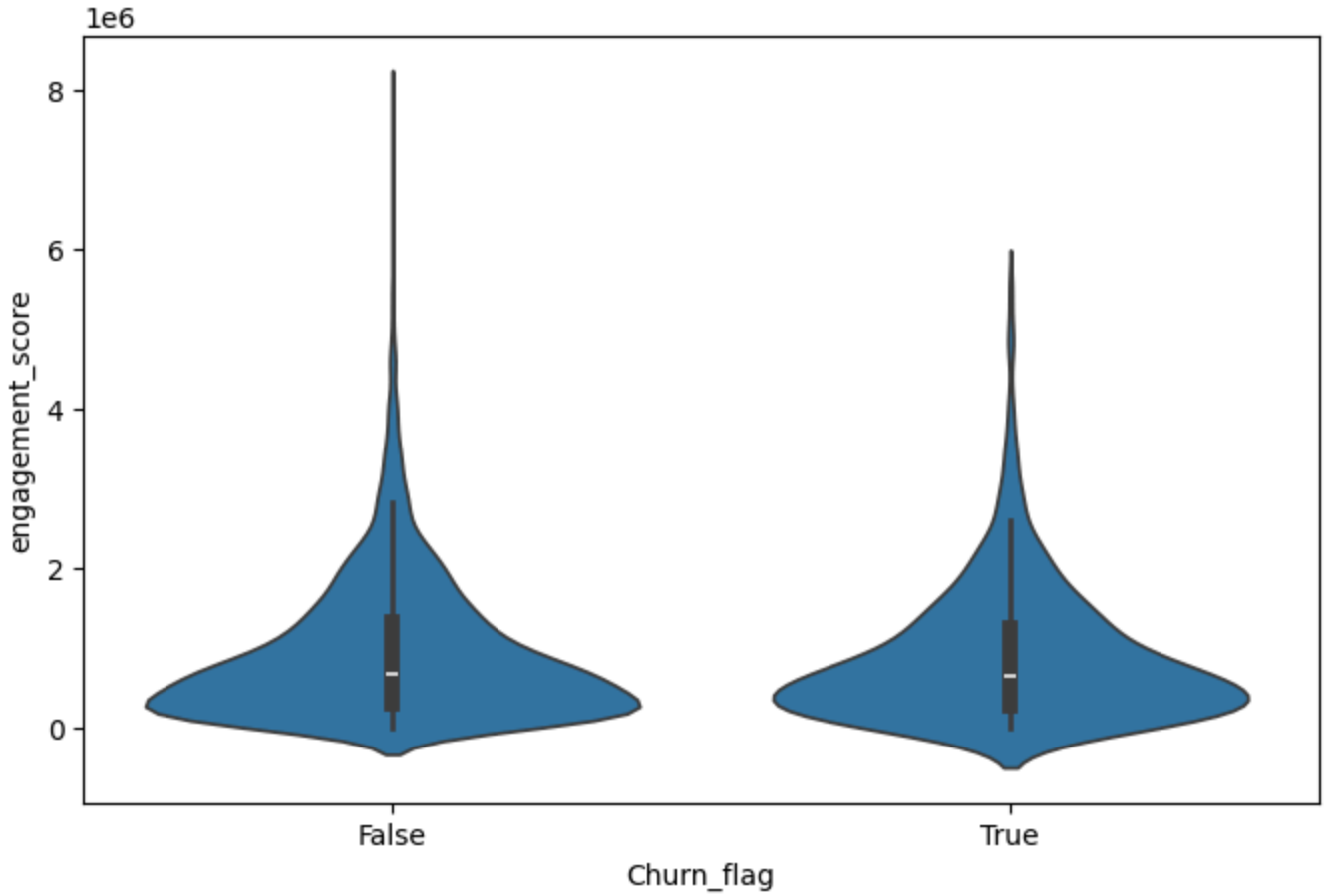

```
#17. Segment vs Usage Duration.

plt.figure(figsize=(8,5))
sns.boxplot(x='Customer_segment', y='Usage_duration_secs', data=df)
plt.show()
```



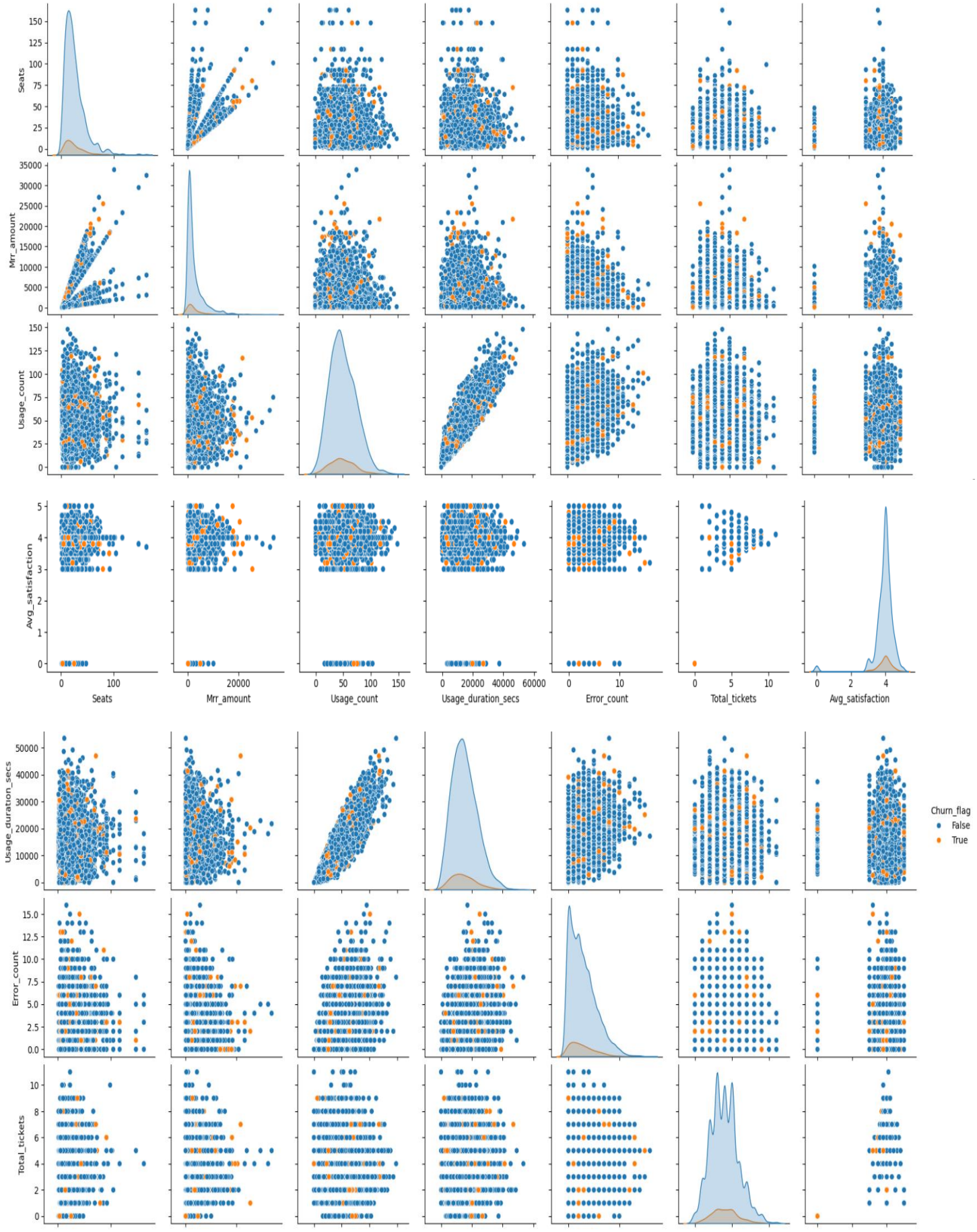

```
#18. Revenue per seat.

df['revenue_per_seat'] = (df['Arr_amount']/df['Seats'])
sns.histplot(df['revenue_per_seat'], kde=True)
plt.show()
```




```
#19. Customer engagement score.

df['engagement_score'] = (df['Usage_count'] *df['Usage_duration_secs'])
plt.figure(figsize=(8,5))
sns.violinplot(x='Churn_flag', y='engagement_score', data=df)
plt.show()
```




```
#20. Pairplot of key behavioral variables.

cols = ['Seats', 'Mrr_amount', 'Usage_count', 'Usage_duration_secs',
        'Error_count', 'Total_tickets', 'Avg_satisfaction']
sns.pairplot(df, vars=cols, hue='Churn_flag')
plt.show()
```


Seats
Mrr_amount
Usage_count
Avg_satisfaction
Usage_duration_secs
Error_count
Total_tickets
Churn_flag
False
True

Conclusions from Python EDA

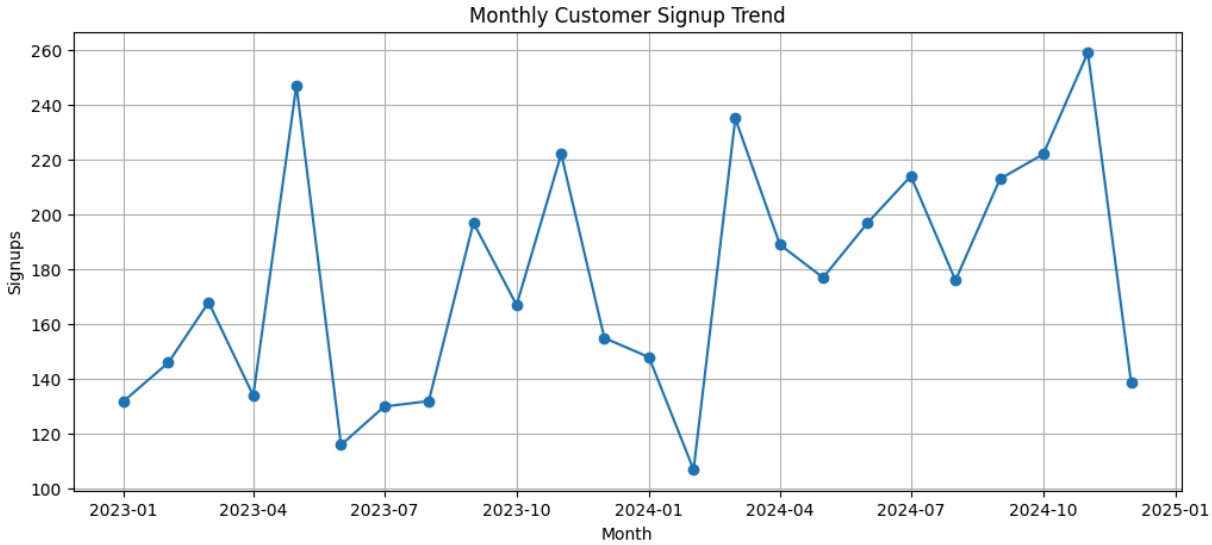
- Customer subscription duration shows noticeable variation, indicating differences in customer retention and lifecycle behavior.
- Signup trends over time highlight fluctuations in customer acquisition, helping identify growth and slower periods.
- Customer movement between subscription tiers reflects upgrade and downgrade behavior, indicating changing customer needs and product adoption.
- Customers with higher usage count and longer platform usage duration tend to be more engaged, suggesting stronger platform dependency.
- Larger organizations with more seats generally demonstrate higher product usage and revenue contribution.
- Increased error counts and support tickets negatively impact customer satisfaction, indicating potential product or service issues.
- Churned customers display different behavioral patterns in terms of engagement, support dependency, and platform activity compared to retained customers.
- Revenue contribution is unevenly distributed, suggesting that a smaller set of customers generates a significant portion of recurring revenue.
- Customer segmentation based on revenue and engagement helps identify high-value customers and potential churn-risk accounts.
- Feature engineering analysis provide meaningful relationships among usage, satisfaction, revenue, support, and churn-related variables, supporting future churn prediction and retention strategies.

VII. Forecasting using ML Modeling

1. Customer Signup Forecasting.

```
# Visualize Historical Trend.

df['Signup_date'] = pd.to_datetime(df['Signup_date'])
monthly_signup = df.groupby(df['Signup_date'].dt.to_period('M')).size()
monthly_signup = (monthly_signup.to_timestamp())

plt.figure(figsize=(12,5))
plt.plot(monthly_signup,marker='o')
plt.title("Monthly Customer Signup Trend")
```

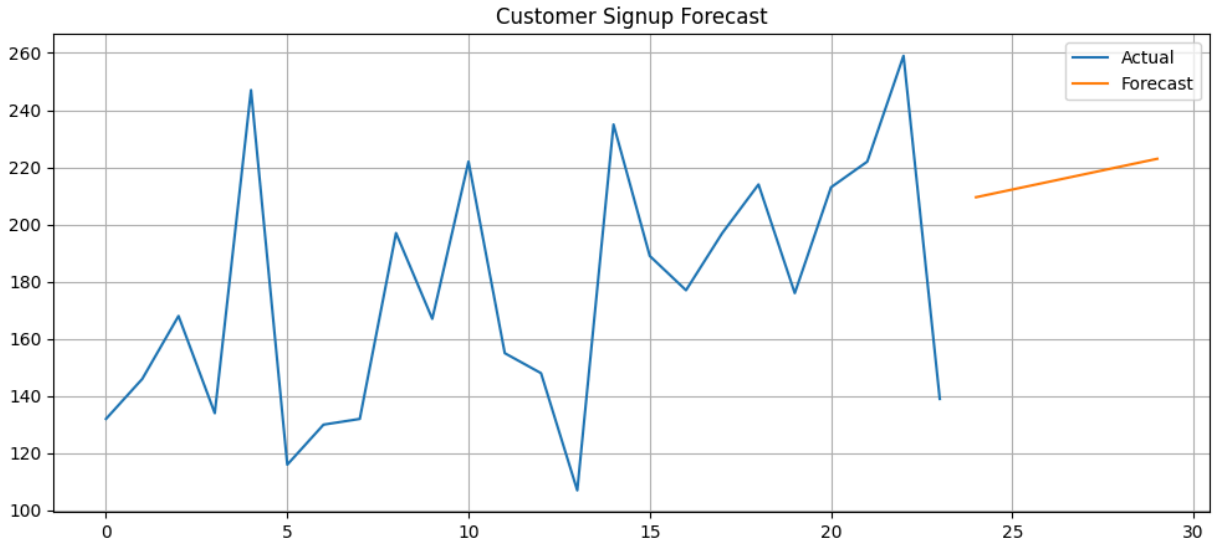
```
plt.xlabel("Month")
plt.ylabel("Signups")
plt.grid(True)
plt.show()
```




```
# Forecast & Plot the Next 6 Months.

from sklearn.linear_model import LinearRegression
import numpy as np

signup_df = (monthly_signup.reset_index())
signup_df.columns = ['Month', 'Signup_Count']
signup_df['Month_Number'] = range(len(signup_df))

X = signup_df[['Month_Number']]
y = signup_df['Signup_Count']

model = LinearRegression()
model.fit(X, y)

future_months = np.arange(len(signup_df),len(signup_df)+6).reshape(-1,1)
forecast = model.predict(future_months)

plt.figure(figsize=(12,5))
plt.plot(signup_df['Month_Number'],signup_df['Signup_Count'],label='Actual')
```

```
plt.plot(future_months, forecast, label='Forecast')
plt.title("Customer Signup Forecast")
plt.legend()
plt.grid(True)
plt.show()
```

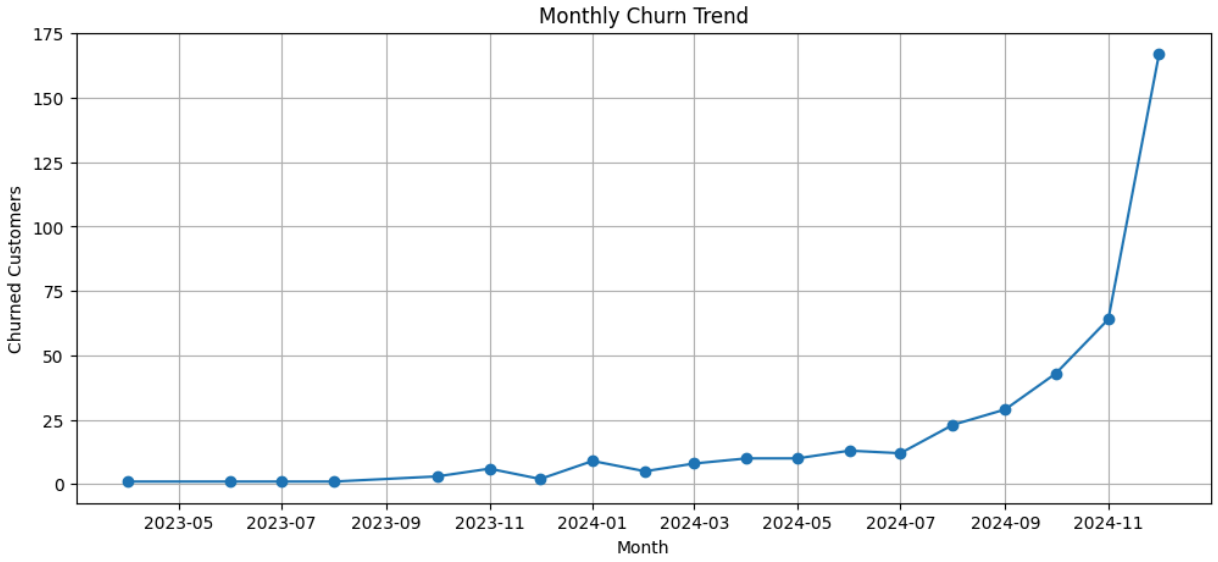
```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```



2. Churn Trend Forecasting.

```
# Plot Churn Trend.

churn_df = df[df['Churn_flag'] == 1]
churn_df['End_date'] = pd.to_datetime(churn_df['End_date'])
monthly_churn = churn_df.groupby( churn_df['End_date'].dt.to_period('M')).size(
monthly_churn = (monthly_churn.to_timestamp())

plt.figure(figsize=(12,5))
plt.plot(monthly_churn, marker='o')
plt.title("Monthly Churn Trend")
plt.xlabel("Month")
plt.ylabel("Churned Customers")
plt.grid(True)
plt.show()
```

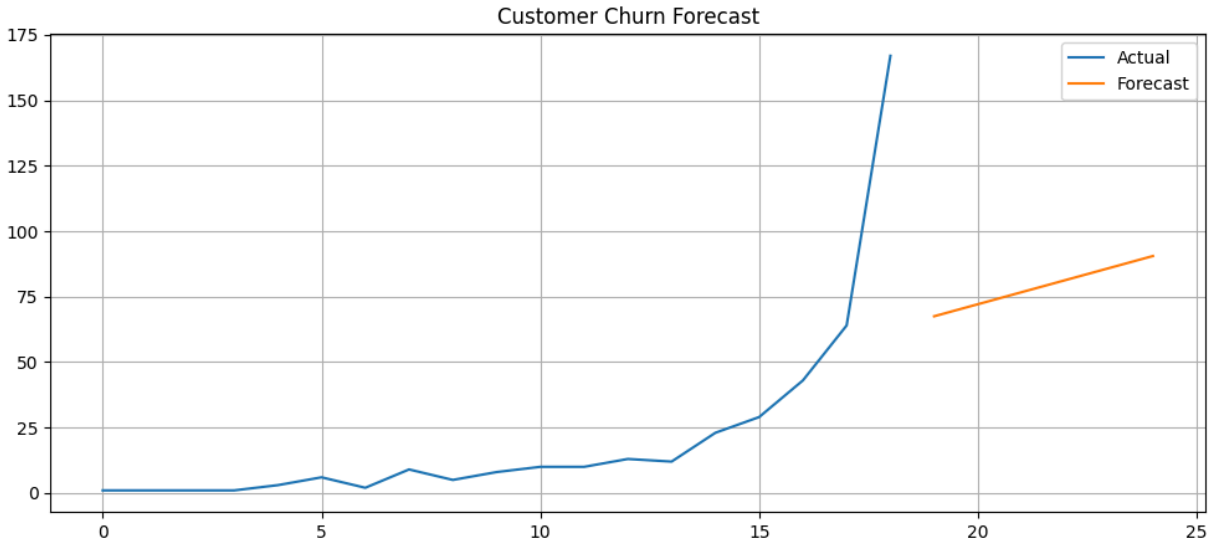
```
/tmp/ipykernel_17977/1919877789.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stab
  churn_df['End_date'] = pd.to_datetime(churn_df['End_date'])
```




```
# Forecast & Plot Churn Prediction.

churn_forecast_df = (monthly_churn.reset_index())
churn_forecast_df.columns = ['Month', 'Churn_Count']
churn_forecast_df['Month_Number'] = range(len(churn_forecast_df))

X = churn_forecast_df[['Month_Number']]
y = churn_forecast_df['Churn_Count']

model = LinearRegression()
model.fit(X, y)

future_months = np.arange(len(churn_forecast_df), len(churn_forecast_df)+6).res
forecast = model.predict(future_months)

plt.figure(figsize=(12,5))
plt.plot(churn_forecast_df['Month_Number'],churn_forecast_df['Churn_Count'],
         label='Actual')
plt.plot(future_months, forecast, label='Forecast')
```

```
plt.title("Customer Churn Forecast")
plt.legend()
plt.grid(True)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```



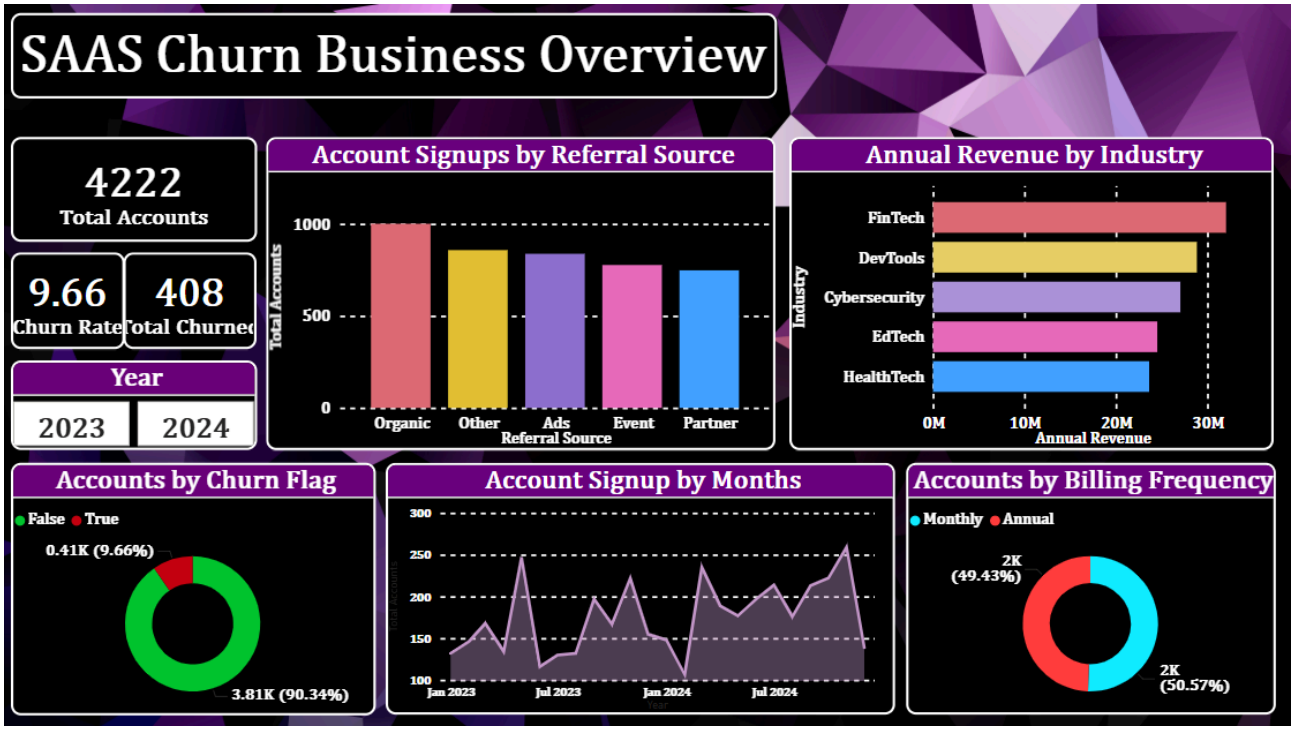
Conclusions from Forecasting Analysis

- Customer signup forecasting indicates future trends in **customer acquisition**, helping estimate expected platform growth over upcoming months.
- Historical signup patterns reveal periods of **growth and fluctuation**, suggesting that customer onboarding may vary over time.
- Churn forecasting helps estimate the **expected number of customers likely to leave the platform**, supporting proactive retention planning.
- Forecasted churn trends highlight the importance of **early customer engagement and retention strategies** to reduce future customer loss.
- Comparing signup and churn trends provides insight into **overall customer growth sustainability** and long-term business stability.
- Variations in signup and churn trends indicate that **customer behavior changes over time**, requiring continuous monitoring.

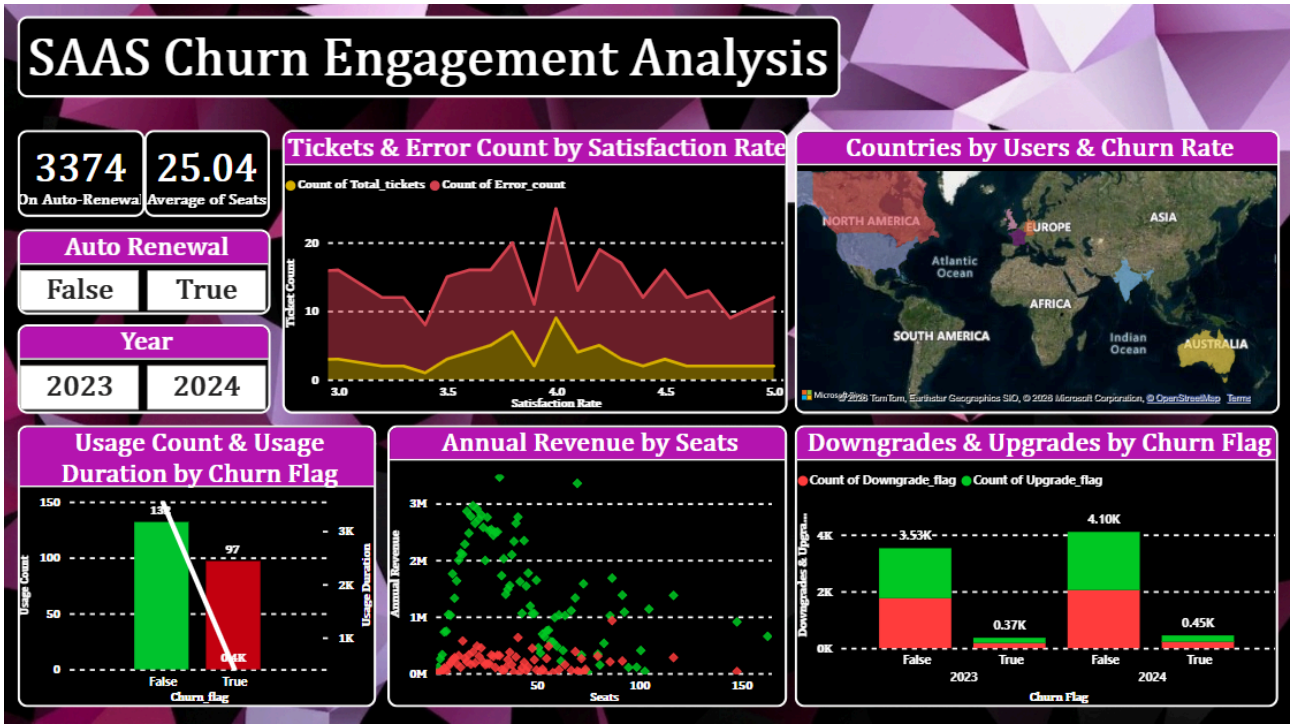
- Forecasting helps identify whether the SaaS business is moving toward **customer expansion or increased attrition risk**.
- Trend-based forecasting provides a data-driven basis for **resource planning, customer success initiatives, and marketing efforts**.
- Historical behavioral trends can be leveraged to improve **customer retention and subscription continuity strategies**.
- Overall, forecasting analysis supports **future business planning, churn reduction, and customer growth optimization** by estimating upcoming customer trends.

Power BI Dashboards

1. SAAS Churn Business Overview – This dashboard provides an overview of overall SaaS business performance, customer acquisition, churn distribution, revenue by industry, and billing behavior. It highlights total accounts, churn rate, signup trends, referral source effectiveness, and customer distribution across billing frequencies.



2. SAAS Churn Engagement Analysis – This dashboard focuses on customer engagement and behavioral patterns by analyzing product usage, satisfaction, upgrades/downgrades, churn behavior, seat usage, and renewal trends. It helps identify how customer activity and engagement influence churn and platform adoption.



3. SAAS Churn Satisfaction & Forecasting Analysis – This dashboard emphasizes customer satisfaction, plan-tier performance, and future business trends through signup and churn forecasting. It provides insights into industry-wise satisfaction, customer growth predictions, and expected churn trends to support proactive business planning and retention strategies.

Analysis of Power BI Dashboarding & Visual Storytelling

- The Power BI dashboards provide a comprehensive view of SaaS business performance by combining customer, revenue, churn, engagement, and forecasting insights in an interactive format.
- KPI cards such as Total Accounts, Churn Rate, ARR, MRR, Usage Hours, and Satisfaction Metrics enable quick monitoring of overall business health.
- Visual storytelling through line charts and trend analysis effectively highlights customer signup growth and churn fluctuations over time.
- Donut charts and bar charts simplify churn distribution, billing frequency, and referral source analysis, making customer segmentation easier to understand.
- Industry-based revenue and satisfaction visualizations help identify high-performing and low-performing business segments for strategic decision-making.
- Engagement-focused visuals such as usage count, usage duration, upgrades, downgrades, and auto-renewal analysis reveal customer behavior patterns linked to retention.
- Geographic mapping of customers and churn rates provides insights into regional performance and customer distribution across countries.
- Forecasting visuals for customer signup and churn trends support future planning by estimating business growth and potential customer attrition.
- The use of interactive slicers and filters (year, churn flag, auto-renewal, country, plan tier, etc.) improves dashboard usability and enables customized analysis.
- Overall, the Power BI dashboards transform raw SaaS data into meaningful visual insights, supporting better business storytelling, churn reduction strategies, customer retention, and revenue optimization.
